Politecnico di Torino

Network-Based Forensic Reconstruction of IoT Device Behaviour: A Simulated Approach

Master in Cybersecurity Engineering

Computer Forensics and Cyber Crime Analysis Project

Eric Ruiz Giménez

Contents

1 Introduction **4**
- 1.1 Context . . . 5
- 1.2 Digital Forensic Problems with IoT . . . 5
  - 1.2.1 Network Forensics . . . 5
- 1.3 Motivation . . . 6
- 1.4 Objectives . . . 6
- 1.5 Scope and Limitations . . . 7
  - 1.5.1 Scope . . . 7
  - 1.5.2 Limitations . . . 8

2 Theoretical Background **8**
- 2.1 IoT Architectures . . . 8
  - 2.1.1 Edge, Fog, Cloud . . . 8
  - 2.1.2 ITU-T Y.2060 . . . 9
- 2.2 Network Protocols . . . 10
  - 2.2.1 Message Queuing Telemetry Transport . . . 10
  - 2.2.2 Constrained Application Protocol . . . 11
  - 2.2.3 HTTP/HTTPS & TLS/SSL . . . 12
- 2.3 Challenges in IoT Forensics . . . 13
- 2.4 IoT Traffic Fingerprinting . . . 14
  - 2.4.1 Device-Level Fingerprinting . . . 14
  - 2.4.2 Behavioral and State Fingerprinting . . . 14
- 2.5 NIST Guidelines . . . 15
  - 2.5.1 Foundational Device Capabilities (NISTIR 8259) . . . 15
  - 2.5.2 Network-Layer Monitoring and MUD (NIST SP 1800-15) . . . 16
  - 2.5.3 Networks of 'Things' Data Flow (NIST SP 800-183) . . . 17
- 2.6 IoT Forensic Frameworks . . . 18
  - 2.6.1 Device and Cloud-Centric Frameworks . . . 18
  - 2.6.2 Network-Centric Frameworks and the Remaining Gap . . . 19
  - 2.6.3 Conclusion and Justification of Proposed Methodology . . . 19

3 Methodology and Setup **20**
- 3.1 Introduction . . . 20
- 3.2 Simulated Network Topology . . . 20
  - 3.2.1 Baseline Behavioural Profile (MUD) . . . 21
- 3.3 Tools . . . 22
- 3.4 Experimental Scenarios . . . 23
- 3.5 Forensic Analysis Pipeline . . . 23

4 Experimental Execution **25**
- 4.1 Simulation Setup and Execution . . . 26
  - 4.1.1 Chain of Custody . . . 26
- 4.2 Scenario 1: Operational Baseline Behaviour . . . 27
  - 4.2.1 Introduction and Objectives . . . 27
  - 4.2.2 Expected Behaviour and Theoretical Baselines . . . 27
  - 4.2.3 Evidence Integrity and Feature Extraction . . . 28
  - 4.2.4 Baseline Validation and Extracted Metrics . . . 28
- 4.3 Scenario 2: Network Reconnaissance and Brute-Force Authentication . . . 32
  - 4.3.1 Introduction and Objectives . . . 32
  - 4.3.2 Expected Behaviour and Theoretical Baselines . . . 32

4.3.3 Evidence Integrity and Feature Extraction . . . 33
4.4 Scenario 3: Unauthorized State Change and Data Exfiltration . . . 36
4.4.1 Introduction and Objectives . . . 36
4.4.2 Expected Behaviour and Theoretical Baselines . . . 37
4.4.3 Evidence Integrity and Feature Extraction . . . 37
4.5 Scenario 4: Botnet Infection . . . 41
4.5.1 Introduction and Objectives . . . 41
4.5.2 Expected Behaviour and Theoretical Baselines . . . 41
4.5.3 Evidence Integrity and Feature Extraction . . . 42
4.6 Cross-Scenario Comparative Analysis . . . 46
4.6.1 The Macro-Statistics and Detection Difficulty Matrix . . . 47
4.6.2 Volumetric Visibility vs. Architectural Stealth . . . 47
4.6.3 The TCP Connection State Tripwire . . . 47
4.6.4 The Degradation of Zero-Trust and MUD Profiles . . . 48

5 Conclusions **49**
5.1 Fulfillment of the Research Questions . . . 49
5.2 Expectations vs. Empirical Reality: The Zero-Trust Paradox . . . 49
5.3 Contributions to the Field . . . 50
5.4 Future Work . . . 50

List of Figures

1 Diagram of an Example IoT System [1] 4
2 Edge, Fog, Cloud Layers architecture [2] 9
3 ITU-T Y.2060 Layered Architecture [3] 10
4 MQTT work schema [4] 11
5 NISTIR8259A capabilities [5]. 15
6 TP Link Camera MUD profile [6]. 17
7 Home IoT Network Design 21
8 Smart Camera MUD Profile. 21
9 22
10 The standardized 4-phase network forensic analysis pipeline applied to all experimental scenarios. 25
11 Scenario 1: Terminal execution demonstrating the SHA-256 hashing and working copy verification. 28
12 Scenario 1: Zeek log extraction of Scenario 1. 28
13 Scenario 1: Smart Camera Bandwidth evolution over experiment time. 29
14 Scenario 1: IAT of Smart Thermostat Keep Alive messages. 30
15 Scenario 1: Empirical interaction heatmap detailing authorized internal network flows. 31
16 Scenario 1: TCP Connection State Distribution. 32
17 Scenario 2: Network Flow Density. 34
18 Scenario 2: TCP Connection State Distribution. 35
19 Scenario 2: Empirical interaction heatmap detailing authorized internal network flows. 36
20 Scenario 3: Throughput change of destination without bandwidth alteration. 39
21 Scenario 3: TCP Connection State Distribution. 40
22 Scenario 3: Empirical interaction heatmap detailing authorized internal network flows. 40
23 Scenario 4 Normal Phase: IAT distribution during the first 300s of experiment. 43
24 Scenario 4 Internal Propagation Phase: Temporal flow density of the 128 unauthorized lateral scanning attempts originating from the infected Smart Thermostat. 44
25 Scenario 4 Volumetric Assault Phase: Sustained UDP flood throughput originating from the compromised thermostat. 44
26 Scenario 4: Network flow density, where a peak can be observed during the network scan. 45
27 Scenario 4: TCP Connection State Distribution. 45
28 Scenario 4: Empirical interaction heatmap detailing authorized internal network flows. 46

List of Tables

1 Authorized Network Flows (MUD Profile) for the Smart Camera 22
2 Authorized Network Flows (MUD Profile) for the Smart Thermostat 22
3 Cryptographic Chain of Custody: SHA-256 hash verification confirming zero data degradation between the original Mininet captures and the working copies analyzed via Zeek. 26
4 Scenario 1: Theoretical Expected Traffic vs. Empirical Zeek Capture Baseline 29
5 Scenario 2: Comparative Traffic Breakdown (Authorized vs. Adversarial) 33
6 Scenario 3: Comparative Traffic Breakdown (Stealth Exfiltration & C2 Routing) 38
7 Scenario 4: Comparative Traffic Breakdown (Thermostat Conscription and 7.47 Mbps Volumetric DDoS Campaign) 42
8 Macro-Statistics and Detection Difficulty Matrix across all experimental scenarios. . . 47

1 Introduction

The Internet of Things (IoT), is a network of interrelated devices that connect and exchange data with other IoT devices and the cloud [1]. These typse of devices are typically embedded with technology such as sensors and software, and can include mechanical and digital machines and consumer objects. They encompass everything from everyday household items to complex industrial tools, making the IoT sector one of the fastest-growing industries today.

With IoT, data is transferable over a network without requiring human-to-human or human-to-computer interactions. In its simplest term, IoT is perceived as a network of billions of devices that can sense, actuate and relay the information to a centralized system. IoT systems gather data from the sensors embedded in IoT devices, which is then transmitted through an IoT gateway for analysis by an application or back-end system. They represent the convergence of Information Technology (data processing and networking) with Operational Technology (hardware and software that detects or causes change through the direct monitoring and/or control of physical devices). The following diagram shows the workflow and the different components that can compose an IoT system.

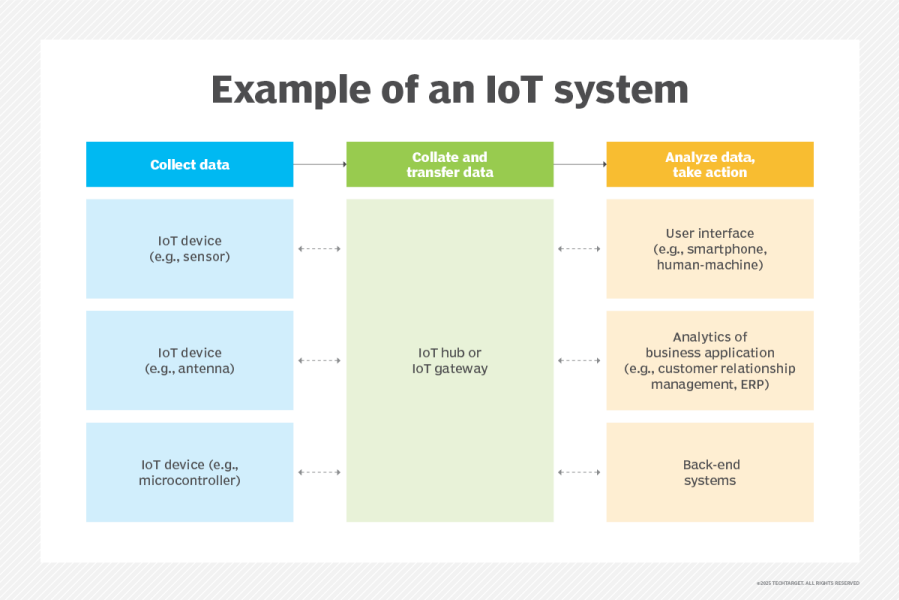

Figure 1: Diagram of an Example IoT System [1]

As a result, the Internet has transitioned from joining end-user nodes to interconnecting physical objects, resulting in a more intelligent object network capable of insightful collaboration and intelligent processing [7]. The capabilities of these type of devices are many such as:

- **Sensing and Actuation:** Devices act as the "edge" between the digital and physical world, either collecting physical data (sensors like temperature or motion detectors) or altering the physical environment (actuators like smart locks or light switches).
- **Computation:** The devices have on-boarded computational power which allow them to perform data handling before transmitting it to reduce the volume of data sent to the cloud.
- **Communication:** They use a network stack, normally low power protocols, to support many-to-many relationships, communicating either device-to-device or device-to-cloud.
- **Autonomy:** They are designed to monitor, automate, and trigger decisions with minimal to no human intervention.

While these capabilities lead to unprecedented efficiency, they also drastically expand the attack surface. Their rapid proliferation has made them frequent targets for cybercriminals, transforming them into a critical and complex focal point for modern digital investigations.

1.1 Context

The widespread adoption of IoT systems has outpaced the implementation of robust security standards, introducing significant vulnerabilities into both consumer and enterprise environments. The Open Worldwide Application Security Project (OWASP) identifies several critical security challenges inherent to the IoT ecosystem [8]. These challenges include among them:

- **Weak, Guessable or Hardcoded Passwords:** Devices are frequently shipped with publicly known default credentials that cannot be easily changed by the end user.
- **Insecure Network Services:** Unnecessary or vulnerable network services running on the device can expose the device to unauthorized access and packet sniffing.
- **Lack of Secure Update Mechanisms:** Many devices the ability to receive firmware updates, or to validate update files cryptographically leaving them permanently vulnerable to discovered exploits.
- **Insecure Data Transfer and Storage:** The lack of strong encryption at rest or in transit allows sensitive operational data to be intercepted or manipulated.

Because of these pervasive security flaws, IoT devices are increasingly weaponized in cyberattacks, serving as entry points for lateral network movement, nodes in distributed denial-of-service (DDoS) botnets, or tools for data exfiltration. Consequently, these devices hold immense forensic value.

1.2 Digital Forensic Problems with IoT

Nevertheless, extracting evidence from these devices exposes the severe inadequacy of traditional digital forensic methodologies. Traditional computer forensics rely on a "dead-box" analysis, a process where the investigator can isolate the device, clone its storages media and then analyse the file system and system logs [9].

IoT devices are totally opposite to this approach. They rely on proprietary hardware, closed source custom firmware and rarely feature standardized physical interfaces such as SATA or USB ports that allow forensically data extraction. Furthermore, due to the necessity of minimizing manufacturing costs and power consumption, they are designed with constrained storage capacities. The vast majority of operational data, state information, and cryptographic keys are held in volatile memory (RAM) [10]. If a device is rebooted, whether intentionally by a malicious actor or accidentally by a user, the evidence is permanently destroyed. Furthermore, even if logging exists on the device's flash memory, the severe storage limitations dictate that these logs are overwritten extremely rapidly in a continuous loop.

1.2.1 Network Forensics

Networked computing, wireless communications and portable electronic devices have expanded the role of digital forensics beyond traditional computer crime investigations [11]. Network Forensics techniques assist in tracking internal and external network attacks by focusing on the devices present and the communication mechanisms applied. The purpose of such an analysis is to explore digital evidence in the network traffic after the occurrence of a suspicious event. In a larger scale environment, it is the ability to sift through gigabytes of the captured network traffic and construct multiple views of

that traffic benefits network security, policy enforcement, and network maintenance personnel. For all these reasons, Cloud Forensics is an emerging branch of Network Forensics.

The objective is the same as physical crime scenes, determining the crime, identifying and analysing the evidence the evidence and establishing the party or parties involved. As for now, there is a standardised method for retrieving evidence from traditional devices but no clear procedures for IoT based investigations, which will require a multi-faceted approach.

iveGn that the IoT devices usually act as inaccessible black boxes with fleeting data, investigators are forced to seek alternative sources of evidence. The defining characteristic of any IoT device is its connectivity, which makes the network traffic it generates its main artefact from where to extract evidence. This traffic is normally easy to analyse since they are single-purpose machines whose patterns are highly predictable and structured.

However, applying traditional network forensics to IoT environments introduces significant technical challenges:

- **Encrypted Payload Barrier:** To protect the user privacy, modern IoT protocols employ encryption techniques. This renders Deep Packet Inspection obsolete. Investigators must rely entirely on encrypted metadata, such as flow duration, inter-arrival times (IAT) and packet lengths.
- **Blurry Network Boundaries:** Not correctly defined perimeter edges in the network are an issue for IoT investigations. The location of evidence effects ease of access, possible connections to other devices (local or cloud based), etc.
- **Time Synchronization:** The existence of multiple sources presents the possibility of different time zone refereces, timestamp interpretations, clock drift issues which can all impact to the ability of generating a unified timeline [12].
- **Semantic gap in Behavioral Translation:** There is a disconnection between raw network data and physical device actions. It is highly complex to map a specific encrypted TCP flow to a real-world event, such as trying to distinguish if a smart lock's network spike indicates a legitimate user unlocking the door or an attacker brute forcing the credentials.
- **Volume and Background Noise:** IoT devices are constantly generating background traffic such as keep-alive messages, heartbeat signals to cloud servers and NTP time synchronizations. Filtering this benign ambient noise to isolate the exact packets corresponding to a forensic event is a major analytical hurdle.

1.3 Motivation

Motivated by these specific challanges, specially the volatility of device-level evidence and the semantic gap present in encrypted communications, this project focuses on developing a robust methodology for IoT behavioral reconstruction. This research aims to validate how passive network forensics can be used to infer physical device actions, bridging the gap between raw network metadata and actionable forensic intelligence.

1.4 Objectives

The main objective of this project is to establish and validate a robust methodology for the behavioral reconstruction of IoT devices using passive network forensics. Due to the previously introduced challenges and limitations, this research focuses on leveraging network evidence to bridge the semantic gap between raw traffic and physical device actions. To achieve this, the project is designed to directly address the following research questions:

1. **Can IoT device activity be inferred solely from network traffic?** By analyzing encrypted packet flows, this project will determine the extent to which physical device actions can be deduced without accessing local device storage.
2. **Which network features best identify device behaviour?** The research will evaluate various traffic metrics such as packet lengths, flow durations, inter-arrival times and specific protocol usage to identify the most reliable indicators if distinct device states.
3. **How accurately can forensic timelines be reconstructed?** By comparing inferred network events against a known baseline of simulated attacks, the project will measure the precision of network-based timeline reconstruction.

To be able to answer these questions, the development of the project is structured around the following actionable objectives:

- **Environment Simulation:** Create a simulated smart home network utilizing virtual IoT devices and lightweight device emulators to generate realistic operational traffic.
- **Threat Simulation and Data Capture:** Establish a baseline of typical device operations, and simulate specific cyberattacks while capturing comprehensive network traffic traces.
- **Feature Extraction and Packet Analysis:** Process the captured traffic abd isolate the behavioural network features using network monitoring tools.
- **Behavioural Classification and Methodology Design:** Develop a clear classification of IoT device behaviors based on their network footprints, and formulate a step-by-step methodology for investigators conducting network-based IoT forensics.
- **Dataset Contribution:** Curate, document, and package the generated network traffic traces into an experimental dataset to be utilized for future IoT forensic research and validation.

1.5 Scope and Limitations

To ensure the feasibility and academic rigor of this research, clear boundaries have been established regarding the environment, methodologies, and technologies evaluated.

1.5.1 Scope

This project strictly focuses on the passive network forensic analysis of IoT environments. The boundaries of the research are defined as follows:

- **Simulated Environment:** The practical experimentation will be conducted within a controlled, simulated smart home network topology. This environment will be constructed using lightweight device emulators and virtualization tools to represent common IoT nodes, such as smart cameras, sensors, and smart plugs.
- **Traffic Analysis:** The research relies entirely on passive network traffic capture and analysis. The extraction of forensic artifacts will be performed using standard network parsing tools, specifically on metadata rather than payload content.
- **Threat Scenarios:** The behavioral reconstruction will focus on a predefined set of common IoT threat scenarios, specifically targeting unauthorized accessand anomalous data flows.
- **Deliverables:** The project will yield a step-by-step forensic methodology, a classification of device behaviors based on network features, and an experimental log dataset generated from the simulated environment.

1.5.2 Limitations

While the simulated approach allows for a highly controlled experimental dataset, it inherently introduces certain limitations to the research:

- **Emulation vs Physical Hardware:** Since the project utilizes virtualized devices and emulators, the generated traffic may lack the complete proprietary background noise and undocumented telemetry that commercial IoT devices present in the real world.
- **Exclusion of Device and Cloud Forensics:** This research operates strictly at the network layer. It explicitly excludes device-level "dead-box" forensics (e.g., memory dumping) as well as cloud forensics (e.g., acquiring data from a vendor's backend).
- **Encrypted Payloads:** The methodology assumes that modern IoT traffic is encrypted (e.g., TLS/SSL). Therefore, the project does not attempt to break encryption, decrypt traffic, or perform Deep Packet Inspection (DPI) on payload data. All inferences are derived from encrypted traffic analysis and flow metadata.
- **Network Scale:** The simulated smart home represents a small-scale, localized network. The findings and the volume of background noise may not directly scale to massive, enterprise-level Industrial IoT (IIoT) deployments or smart city infrastructures.

2 Theoretical Background

Establishing a foundational understanding of the underlying technologies and review the current state of the art in digital investigations is crucial to be able to develop a methodology for IoT behavioural reconstruction. This section provides the theoretical framework necessary to contextualize the proposed research, mapping the technical landscape of connected devices and identifying the existing gaps in current forensic methodologies.

2.1 IoT Architectures

To successfully analyse the network behaviour of an IoT device, it is first necessary to understand the structural paradigm in which it operates. Contrary to the traditional network based on standard endpoints (desktop computers and servers), the IoT ecosystem is highly distributed and heterogeneous. Academic consensus generally model the IoT architecture across three primary tiers: the Edge (or Perception) Layer, the Fog (or Network/Gateway) Layer, and the Cloud (or Application) Layer [2]. To address increasing complexity, standard bodies such as the International Telecommunication Union (ITU-T) have formalized a four-layer model (Device, Network, Service Support and Application layers). [13].

Understanding this layered architecture is critical for digital forensics as it defines where data is generated, how it is encapsulated in transit and where investigators can feasibly deploy network capture tools.

2.1.1 Edge, Fog, Cloud

This hierarchical structure addresses the challenges of IoT by distributing processing and storage responsibilities across these layers. This allows the IoT system to provide locally real-time responsiveness while leveraging the vast analytical power of the cloud.

- **Edge Layer:** Lowest tier of the architecture, consisting only of hardware deployed in the environment. Includes elements such as sensors, which gather data, and actuators, which perform physical actions. This type of devices have minimal processing power as well as a highly volatile

memory. Their objective is to interact with the physical world but rely on upper layers for complex processing and long-term storage.

- **Fog:** Acts as the bridge between both layers, the local smart environment and the wider internet. Consists of network gateways, local routers and edge computing nodes [14]. Since Edge devices use low-power, non-IP communication protocols, the gateway has responsibility of translating these signals into standard IP traffic for internet transit. Furthermore, it often performs data normalization, ensuring that sensor formats are unified before transmission. This point is vital for digital forensics since it offers visibility of all localized device behaviours and its outbound communications before they are integrated to the cloud.
- **Cloud:** Most widely adopted architecture across industries. A cloud platform consists of distributed servers and storage resources accessible over the internet. These systems provide virtually unlimited computing and storage, elastic scalability, centralized data aggregation and high data analytics capabilities.

The most important element for digital forensics in this architecture is the Fog layer since it is the translator and the routing point for the smart environment with the cloud.

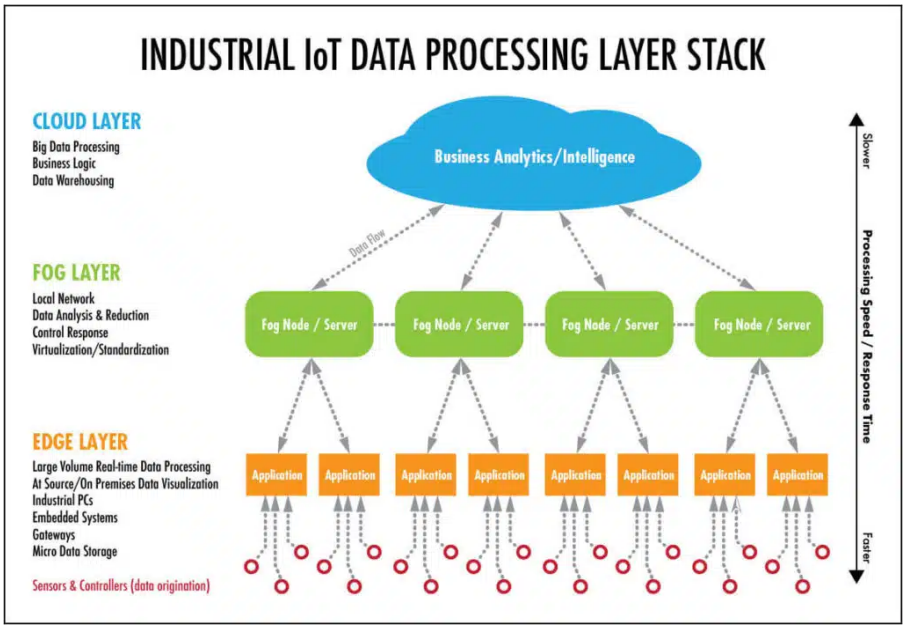

Figure 2: Edge, Fog, Cloud Layers architecture [2]

2.1.2 ITU-T Y.2060

ITU framework focused on a four vertical stack supported by two horizontal management and security capabilities that span accross all layers.

The Four-Layer Architecture

1. **Application Layer:** The top tier where specific IoT applications reside. It interacts with the lower layers to consume processed data and provide services to the end-user.
2. **Service Support and Application Support Layer:** Acts as a middleware. It provides common capabilities used by many applications (Service Support),and capabilities tailored to a particular industry or application type (Specific Support).

3. **Network Layer:** Responsible for communication and connectivity. It handles the Networking Capabilities (routing and transport) and Access Capabilities (connecting devices to the core network).
4. **Device Layer:** The physical foundation. It includes the "Things" (sensors and actuators). Devices can interact with the network directly or through a **Gateway** if they use low-power, non-IP protocols.

Cross-Layer Capabilities

A key feature of the ITU architecture is that it identifies two functions that must exist at every single level of the stack:

- **Management Capabilities:** Covers fault management, configuration, accounting, performance, and security management (FCAPS) across the entire system.
- **Security Capabilities:** Ensures data privacy, device authentication, and communication integrity at every point where data is handled or moved.

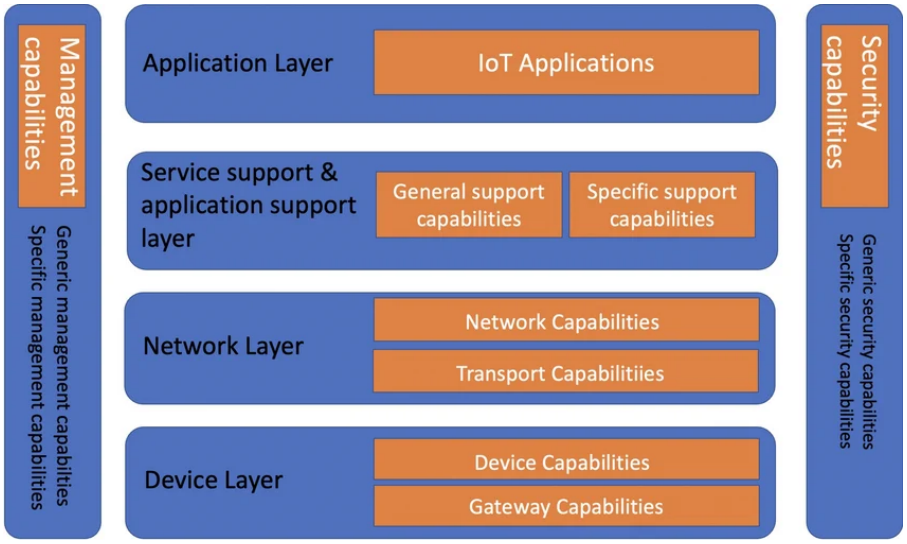

Figure 3: ITU-T Y.2060 Layered Architecture [3]

The ITU model also formalizes the Gateway as a specific functional entity within the Device Layer. This confirms the claim that the Fog/Gateway layer is the most important point for digital forensics, as it is the first point where device specific data is aggregated and managed.

2.2 Network Protocols

2.2.1 Message Queuing Telemetry Transport

Message Queuing Telemetry Transport (MQTT) was originally developed by IBM in 1999 and it is currently maintained as an OASIS standard. It is one of the most widely adopted messaging protocols in the IoT ecosystem [15]. It is specifically designed for constrained devices operating on low-bandwidth, high-latency or unreliable networks. It is the standard for smart home sensor communication.

Contrary to web protocols such as HTTP, MQTT operates on a **Publish/Subscribe** architectural model. IoT devices do not communicate directly with one another, all communications are routed through a central server known as **MQTT Broker**, which typically resides at the Fog layer or in the

Cloud. Devices publish messages called topics and the rest of devices can subscribe to these topics to receive updates.

The protocol runs over the TCP/IP stack and exhibits highly specific identifiable network footprints:

- **Standardized Ports:** By default, unencrypted traffic operate over TCP port 1883. While encrypted (MQTT over TLS/SSL) uses port 8883. Identifying persistent TCP streams on these ports is the primary method for isolating MQTT traffic from background network noise.
- **Keep-Alive Mechanisms:** To maintain the persistent TCP connection without constantly sending data, MQTT clients periodically transmit `PINGREQ` (Ping Request) packets to the broker, which responds with a `PINGRESP` (Ping Response). These periodic and uniformly sized packets establish a critical baseline of "normal" device behavior. An interruption in this keep-alive rhythm often indicates device failure, a reboot, or a targeted denial-of-service attack.
- **Quality of Service (QoS) Flows:** MQTT defines three QoS levels (0: At most once, 1: At least once, 2: Exactly once) that dictate the reliability of message delivery. Higher QoS levels require a multi-step acknowledgment handshake (e.g., `PUBACK`, `PUBREC`, `PUBREL`). For an investigator, observing these acknowledgment sequences in the traffic flow provides insight into the importance and type of data being transmitted, even if the payload is encrypted [16].

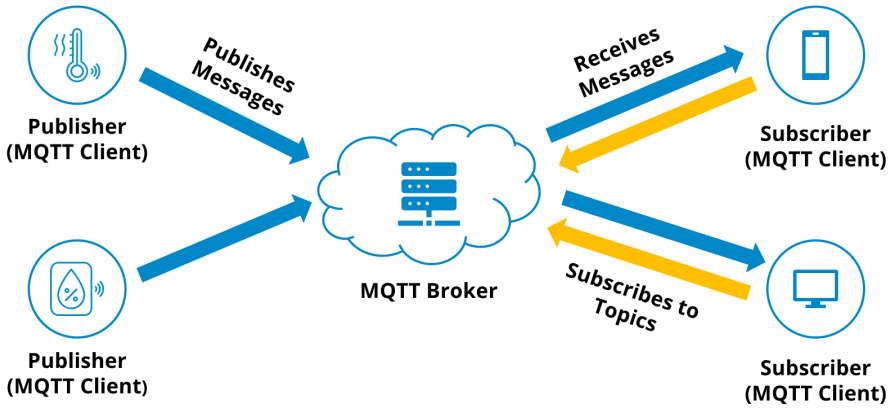

Figure 4: MQTT work schema [4]

Modern security practices strongly mandate the use of MQTT over TLS (port 8883). Consequently, modern IoT forensic methodologies cannot rely on reading the payload. Investigators should use metadata such as frequency of publish packets, the size of the encrypted payloads and the keep-alive intervals to infer the device's physical state.

2.2.2 Constrained Application Protocol

The Constrained Application Protocol (CoAP) was developed by the Internet Engineering Task Force (IETF) to serve as a lightweight, specialized alternative to HTTP for constrained devices [17]. Operating on a traditional client-server RESTful architecture, CoAP allows IoT devices to utilize familiar web methods (GET, POST, PUT, DELETE) while maintaining an extremely low overhead suitable for low-power microcontrollers.

The most significant distinction between CoAP and MQTT is the transport layer. While traditional HTTP and MQTT rely on the connection-oriented Transmission Control Protocol (TCP), CoAP is designed to operate primarily over the connectionless User Datagram Protocol (UDP). This architectural choice drastically alters the network artifacts available for behavioral reconstruction:

- **Connectionless Traffic Analysis:** By default, CoAP operates over UDP port 5683, while encrypted CoAP traffic utilizes UDP port 5684. Because UDP lacks the traditional three-way handshake (SYN, SYN-ACK, ACK) and connection teardown (FIN) of TCP, investigators cannot rely on standard network flow boundaries to determine when a device session begins or ends. Instead, forensic analysts must look for specific temporal clusters of UDP packets to infer interaction.
- **Application-Layer Reliability Mechanisms:** To compensate for the unreliability of UDP, CoAP implements its own lightweight reliability mechanisms directly within the message header. CoAP frames are categorized into four distinct types: Confirmable (CON), Non-confirmable (NON), Acknowledgement (ACK), and Reset (RST). From a network analysis perspective, observing the ratio and timing of CON to ACK packets provides excellent behavioral indicators. For example, a sudden spike in unacknowledged CON messages may indicate a device experiencing a network failure, a localized jamming attack, or a disrupted cloud backend.
- **Message IDs and Tokens:** Because UDP packets can arrive out of order, CoAP utilizes Message IDs for detecting duplicates and matching ACKs and Tokens for matching asynchronous requests and responses. These identifiers act as critical metadata threads, allowing them to form together a chronological sequence of requests and responses even when the surrounding network environment is highly congested.

Modern security standards mandate that CoAP payloads should be encrypted. CoAP utilizes Datagram Transport Layer Security (DTLS) on port 5684. Once DTLS is established, the specific RESTful commands are obscured. Consequently, the forensic methodology must pivot entirely to metadata analysis as with MQTT. Some examples of techniques are correlating the size of the DTLS payload, the direction of the traffic, and the frequency of the UDP datagrams to classify the physical action the device is executing.

2.2.3 HTTP/HTTPS & TLS/SSL

The traditional Hypertext Transfer Protocol (HTTP) and its secure counterpart (HTTPS) remain fundamental to the broader IoT ecosystem. HTTP relies on a client-server RESTful architecture operating over the Transmission Control Protocol (TCP). In IoT environments, HTTP is normally used by high-bandwidth devices, administrative web interfaces and the critical Fog layer gateways and Cloud layer vendor backends.

As the rest of protocols, HTTPS is mainly used for security concerns. This protocol version obscures he payload and the specific URL paths accessed by the device. To reconstruct device behaviour network forensic methodologies must pivot to analysing the cryptographic handshakes and contextual flow metadata:

- **Server Name Indication (SNI):** During the initial TLS handshake, before the encrypted tunnel is fully established, the client typically transmits a `ClientHello` message in cleartext. This message frequently contains the Server Name Indication (SNI) extension, which reveals the plaintext domain name the device is attempting to contact. The SNI is arguably the most valuable artifact in encrypted traffic, as it immediately identifies the device's manufacturer, its intended cloud backend, and often its specific function.
- **Flow Duration and Throughput Profiling:** Because the RESTful commands are encrypted, investigators must infer the device's action by analyzing the shape of the TCP flow. A short, low-byte TCP flow likely represents a routine heartbeat or authentication check with the cloud. Conversely, a prolonged TCP session with massive upstream byte transfer strongly indicates active video streaming or data exfiltration.

- **TLS Fingerprinting:** Because IoT devices utilize highly specific, embedded operating systems and libraries (such as OpenSSL or mbedTLS), they negotiate TLS connections in highly predictable ways. By analyzing the specific combination of cipher suites, supported elliptic curves, and TLS extensions offered during the `ClientHello`, investigators can create a cryptographic fingerprint of the device. This allows analysts to definitively identify the type of IoT device generating the traffic, even when the traffic is fully encrypted and background noise is high.

2.3 Challenges in IoT Forensics

As previously introduced in the previous section, conducting digital forensics within the IoT ecosystem presents a unique set of obstacles that make classical device based methodologies ineffective. To successfully investigate these environments, analysts must thoroughly understand these limitations and apply workarounds mainly based on network forensics. The primary identified challanges are listed below:

- **Ephemeral Data and Constrained Storage:** The majority of operational data and cryptographic keys are held in volatile memory (RAM) and are permanently lost upon a device reboot. Furthermore, due to constrained storage capacities, any existing flash memory logs are rapidly overwritten in a continuous loop.
- **Proprietary Hardware and Interfaces:** Devices rely on closed-source custom firmware and lack standardized physical data extraction interfaces (such as SATA or USB ports), acting as inaccessible "black boxes".
- **The Encrypted Payload Barrier:** To protect user privacy, modern IoT protocols implement encryption methods. This results in Deep Packet Inspection (DPI) being obsolete, as investigators cannot read the payload contents.
- **Semantic Gap in Behavioral Translation:** There is a fundamental disconnect between raw, encrypted network data and physical device actions, making it difficult to map a TCP flow to a real-world event (e.g., a smart lock opening).
- **Volume and Background Noise:** IoT devices are highly chatty, constantly generating benign ambient noise such as keep-alive messages, NTP synchronizations, and heartbeat signals to cloud servers.
- **Time Synchronization and Blurry Boundaries:** The existence of multiple data sources with varying time zones, clock drifts, and blurry network perimeters complicates the generation of a unified chronological timeline.
- **Protocol Heterogeneity and Network Fragmentation:** Unlike traditional IT networks that primarily rely on standard TCP/IP over Ethernet or Wi-Fi, the IoT ecosystem is highly fragmented. A single smart home may simultaneously utilize different protocols to implement their services. This requires investigators to deploy multiple specialized sniffing tools and gateways to capture the full spectrum of device communications, complicating the unified capture process.
- **Cloud Dependency and Data Fragmentation:** Many IoT architectures rely heavily on cloud processing. Consequently, local device-to-device interactions are frequently routed out to vendor-controlled external servers before executing a local action. Local network captures rely on analyzing encrypted outbound streams rather than direct, localized command-and-control traffic.

These constraints demonstrate that traditional forensics techniques and procedures are incompatible with the IoT ecosystem. Because investigators cannot reliably extract persistent data from physical edge devices, nor easily decrypt in transit communications, the network gateway emerges as the most critical vantage point for evidence acquisition.

However, the presence of an encrypted payload leads on an insufficient capturing of network traffic. To successfully reconstruct a chronological timeline of events, investigators must pivot from content-based analysis to context-based analysis. This requires the development and application of advanced methodologies capable of identifying specific devices which objective is to guess the state of the device based on the encrypted network flow and its metadata. This is known as IoT traffic fingerprinting.

2.4 IoT Traffic Fingerprinting

To bridge the gap between encrypted network flows and physical device events, investigators use a methodology known as IoT traffic fingerprinting. **Fingerprinting** is the process of passively observing network traffic to extract a unique set of features or metadata (a fingerprint) that can reliably identify a specific device, its operating system, or its current physical state [18].

In traditional IT environments, fingerprinting often relies on deep packet inspection (DPI) or active scanning. An example of active scanning can be sending tailored ICMP or TCP SYN packets to observe the target's response. However, because IoT devices are resource-constrained and easily disrupted by active scanning, and their payloads are predominantly encrypted, modern IoT fingerprinting relies exclusively on the passive statistical analysis of traffic flows.

IoT fingerprinting can be divided into two distinct phases: Device-Level Fingerprinting and Behavioral and State Fingerprinting.

2.4.1 Device-Level Fingerprinting

The first objective when capturing network traffic must be to identify the devices present in the environment. Device fingerprinting relies on the highly repetitive and single-purpose nature of IoT hardware. In general, most IoT devices have different hardware components as well as forming part of different cloud architectures. For this reason, IoT network footprints are usually highly distinct [19]. To classify device types, investigators analyze:

- **Protocol Signatures:** Identifying the specific combination of protocols used. The use of a port or another indicates which protocol is implemented by the device.
- **Initialization and DNS Patterns:** When IoT devices boot or reconnect to a network, they frequently perform specific DNS queries to vendor-controlled domains for time synchronization or for cloud registration). Extracting these plaintext queries from DNS logs (typically UDP port 53) provides immediate, high-fidelity device attribution.
- **TLS Handshake Metadata:** If TLS is used, as discussed in Section 2.1, extracting the Server Name Indication (SNI) and mapping the supported cipher suites negotiated during the `ClientHello` phase allows investigators to uniquely identify the device's embedded operating system [18].

2.4.2 Behavioral and State Fingerprinting

Once the devices involved in the capture are identified, the focus turns to behavioral reconstruction. Behavioral fingerprinting seeks to relate variations in the network flow to specific physical actions. For example, differentiating between a camera standing by, actively recording, or being rebooted. Because the payloads are encrypted, this identification relies heavily on flow-level statistics. The most crucial characteristics utilized for behavioral fingerprinting include:

- **Packet Length and Size Distribution:** Specific actions trigger specific payload sizes. For example, a smart lock communicating an "unlock" command will consistently generate a highly specific, small packet size compared to a camera streaming a variable bitrate video.

- **Inter-Arrival Time (IAT):** IAT measures the time elapsed between consecutive packets in a flow. IoT devices often exhibit highly periodic IATs due to automated heartbeat messages. A sudden, dense clustering of packets typically signifies a triggered physical event or an ongoing cyberattack.
- **Flow Volume and Duration:** Analyzing the total bytes transferred over a specific window of time helps categorize the device's current state. For example, a baaseline state may show 5 KB transferred per minute, while an active data exfiltration or firmware update event may spike to several megabytes per minute.

By establishing a baseline of these statistical features during normal operation, investigators can create and define a classification of the known states for each device. When an anomaly or malicious event occurs, the resulting traffic spike can be matched against this classification, allowing the investigator to reconstruct a timeline of the device's physical behaviour without ever decrypting the underlying data.

2.5 NIST Guidelines

To successfully apply the traffic fingerprinting and behavioural reconstruction methodologies previously discussed, a "ground truth" must be established. This ground truth is mostly based on reconstructing a forensic timeline, which basically relies on the ability to distinguish between normal operational traffic, user-triggered physical events, and malicious anomalies. To define these baselines, the cybersecurity sector heavily rely on the standards and publications made by important institutions. A great example to follow, are the baselines established by the National Institute of Standards and Technology (NIST).

NIST has published several highly influential frameworks that dictate how IoT devices should be designed, how they should behave on a network, and how their traffic has to be managed. For the purposes of network-based IoT forensics, NIST guidelines are particularly relevant: Foundational Device Capabilities and Network-Layer Behavioral Whitelisting.

2.5.1 Foundational Device Capabilities (NISTIR 8259)

The NIST Interagency Report (NISTIR) 8259 series [20] provides guidance for manufacturers and their supporting third parties as they conceive, design, test, sell and support IoT devices across their spectrum of customers. From a forensic perspective, the most critical mandate within this framework is the IoT Device Cybersecurity Capability Core Baseline [5].

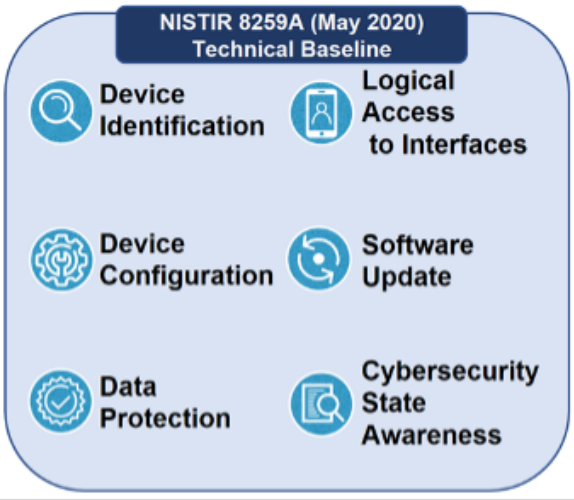

Figure 5: NISTIR8259A capabilities [5].

NIST dictates that an IoT device should have a strictly defined, predictable set of network interactions. The document outlines the capability of IoT devices to restrict and manage local and network accesses. For example, it defines that an IoT device should only communicate with specific authorized endpoints using specific protocols. By mandating that manufacturers document these expected behaviors, NIST implicitly validates the core premise of network-based IoT forensics. This means that any deviation from this documented predictable network footprint strongly indicates a physical state change, a configuration error, or a security compromise.

Furthermore, the 8259A baseline mandates that devices must implement a **Device Identification** method such as a logical identifier like a MAC address or UUID. This is important for the forensic investigator since the identification of this unique identifier is the first step in the timeline reconstruction.

There are some other capabilities that must be defined that can be important for a forensic investigation:

- **Data Protection:** NIST mandates that manufacturers must use cryptography to protect data in transit. For this reason, Deep Packet Inspection is not the solution, making feature extraction necessary.
- **Software Update:** Devices must be able to receive Over-The-Air firmware updates. This has a highly distinct network footprint since the device must perform a DNS request, followed by a massive TCP data flow.
- **Cybersecurity State Awareness:** The device should be able to report its status and special events such as errors, crashes, unauthorized access attempts to the cloud or local admin. This means that devices will automatically generate automated network traffic when something goes wrong. The identification or monitoring of these notifications is important since a spike in State Awareness traffic can indicate the existence of an ongoing attack.

2.5.2 Network-Layer Monitoring and MUD (NIST SP 1800-15)

Device logs are unreliable as previously discussed in Section 2.2. For this reason, NIST Special Publication 1800-15 [21] defines the concept of Manufacturer Usage Description (MUD) specificagtion for IoT devices. The objective is to ensure that manufactures can indicate the network communications that a device requires to perform its intended function. When MUD is used, the network will automatically permit the IoT device to send and receive only the traffic it requires to perform as intended, and the network will prohibit all other communication with the device, increasing the device's resilience to network-based attacks.

This standard specifies a data model, a JSON-formatted file, to describe a set of access control entries representing allowed communications of an IoT device on the network. It also defines in-band mechanisms by which an IoT device can signal to the network supplying the MUD model that indicates what access and protection the device needs. Devices typically broadcast the URL of their MUD file in plaintext during the initial network boot sequence. The following image is an example of the MUD profile for a TP Link Camera.

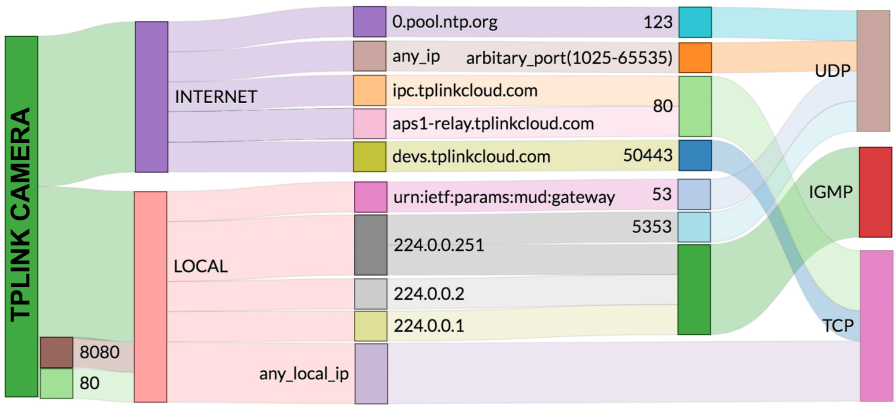

Figure 6: TP Link Camera MUD profile [6].

For example, it specifies that the device communicates with *0.pool.ntp.org* over UDP port 123 to use the Netwrok Time Protocol (NTP) in order to keep the camera's internal clock synchronized. Or its use of TCP port 80 via *ipc.tplinkcloud.com* and *aps1-relay.tplinkcloud.com* possibly for basic API check-ins, relay negotiations or unencrypted telemetry with the manufacturer's cloud.

For a forensic investigator, the principles behind MUD are foundational for behavioral reconstruction:

- **Establishing the Normal Baseline:** MUD profiles define the exact network features (destination IP addresses, transport protocols, and port numbers) that constitute normal behaviour.
- **Anomaly Detection as an Event Trigger:** If an investigator analyzing a PCAP file observes traffic that violates these NIST-recommended baseline, it can be used as a high-fidelity timestamp for a forensic event.
- **State Transitions:** By understanding the approved architectural flow of a smart home environment as outlined by NIST, investigators can map sudden bursts in compliant traffic to specific physical state transitions or possible attacks.

2.5.3 Networks of 'Things' Data Flow (NIST SP 800-183)

To accurately reconstruct a forensic timeline, an investigator must understand the underlying science of how IoT data moves. NIST Special Publication 800-183 [22] establishes the foundational primitives of IoT architectures, modeling the ecosystem using five core components: Sensors, Aggregators, Communication Channels, eUtilities, and Decision Triggers.

For forensic timeline reconstruction, this framework is vital because it formalizes the chronological flow of IoT events. It dictates that a physical event begins at the Sensor, is encapsulated into packets traveling via a specific Communication Channel (the network), is routed by an Aggregator (the gateway), and ultimately results in a Decision Trigger (a physical actuation). Investigators can use this theoretical data flow model to map chronological packet sequences in a PCAP file directly back to the physical sequence of events occurring in the smart home.

Finally, while NIST guidelines are designed for security enforcement, risk management following a manufacturer oriented point of view, they provide the theoretical base required for forensic analysis. They confirm that IoT network traffic is intended to be highly structured and restricted, ensuring that the network feature extraction methods proposed in this research can reliably isolate specific device behaviours from benign background noise.

2.6 IoT Forensic Frameworks

As it has been previously been highlighted in previous sections, the academic and cybersecurity communities have recognized the inadequacy of classical forensic models. Consequently, numerous researchers have proposed specialized IoT forensic frameworks designed to navigate the highly distributed nature of these ecosystems. This subsections aims to review the state-of-the art frameworks to understand the current capabilities of digital investigators and to identify the gaps that this research seeks to address.

Current IoT forensic existing frameworks can generally be categorized into three approaches: Device-Centric, Cloud-Centric and Network-Centric model.

2.6.1 Device and Cloud-Centric Frameworks

First steps to formalize IoT investigations specially relied on extending traditional computing methodologies to smart devices. Forensic-Aware IoT (FAIoT) model [23], is a clear example of this case. This framework advocates for a secure by design approach, arguing that forensic readiness must be directly built into the IoT infrastructure. In this schema, edge devices do not merely execute commands, they continuosly generate cryptographically signed operantional logs. Then, using Public Key Infrastructure (PKI) and secure hashing algorithms, devices immediately push these authenticated logs to a centralized and immutable repository known as the Secure Evidence Cloud (SEC).

From the theoretical point of view, FAIoT directly neutralizes the hardware limitation of IoT devices by continuously offloading logs to the SEC. This ensures that data is preserved even if the physical edge device is rebooted, destroyed or compromised by an attacker. In addition, the SEC acts as a tamper-proof vault accessible to law enforcement via read-only APIs (non-repudiation and integrity). This ensures that the extracted timeline of events for an investigation has been not altered, satisfying the legal requirements for digital evidence admissibility.

Another example is the multi-tiered framework proposed by Kebande and Ray [24]. This schema divides the investigation into specific zones (the Edge/Device Zone, the Network Zone, and the Cloud Zone), suggesting that investigators should acquire evidence from the physical device's flash memory or subpoena the vendor's cloud servers. This framework adapts established incident response standards such as ISO 27043 into a layered methodology, where the investigation landscape is divided into three distinct operational zones.

- **Edge Zone:** Groups the devices where investigation relies on classical hardware forensics attempting to extract residual state data from volatile memory (RAM), dump priopetary firmware via physical ports or analyze the limited local flash storage.
- **Network Zone:** Represents the communications channels and localized routing hardware (Smart Home Gateway). Evidence gathering focuses on intercepting in-transit data, capturing network packet traces (PCAPs) and analyzing protocol flows to reconstruct device-to-device or device-to-cloud communications.
- **Cloud Zone:** Ultimate centralized repository for device telemetry. This zone gathers vendor controlled data centers and APIs. The forensic efforts related to this zone involve acquiring long term historical data, user account activities, aggregated event logs...

This multilayered framework does noy treat the IoT incident as a single isolated event, since it forces the analyst to map the entire evidence trail from the physical trigger at the Edge, passing through the routing mechanisms at the network zone until the final persistent storage in the Cloud.

While theoretically sound, these frameworks suffer from practical limitations in real world scenarions:

- **Manufacturer Reliance:** Frameworks like FAIoT require manufacturers to proactively build forensic logging into the devices, a practice that is economically and computationally prohibitive for IoT devices in general.
- **Jurisdictional and Cloud Barriers:** Relying on the Cloud Zone requires legal consent from international cloud providers like AWS or TP-Link servers among others. This is a process that is exceedingly slow and often impossible for localized incident response.
- **Data Volatility:** As established in previous sections, by the time an investigator physically reaches the Edge Zone to attempt memory extraction, the volatile RAM containing the cryptographic keys and recent event states has typically been overwritten or lost to a reboot.

2.6.2 Network-Centric Frameworks and the Remaining Gap

Recognizing the physical and legal barriers of the Device and Cloud zones, modern forensic research has increasingly pivoted to the Network zone. Network-centric frameworks operate on the premise that local gateway is the only reliable and accessible point for evidence acquisition, since it is the chokepoint for all IoT communications. Because edge devices must route their traffic through this centralized hub to reach the wider internet or interact with peer devices, the gateway becomes the most reliable and accessible point for continuous evidence acquisition.

However, while the current body of literature broadly acknowledges the necessity of network packet capture (PCAP) in IoT investigations, a significant methodological shortcoming persists. Existing frameworks overwhelmingly focus on *Device Identification*. This is the static process of fingerprinting a device's specific make, model, and operating system. While cataloging the hardware is a necessary preliminary step, these frameworks generally fail to address *Behavioral Reconstruction*. They lack the capability to dynamically infer what the device is physically doing at a specific millisecond, such as distinguishing between a smart camera sitting idle, actively streaming video, or undergoing a malicious remote reboot. Furthermore, many legacy network forensic models still fundamentally rely on the assumption of plaintext communications, heavily utilizing Deep Packet Inspection (DPI) to simply read cleartext HTTP or MQTT payloads to determine device actions.

This reliance on DPI renders these legacy frameworks effectively obsolete in contemporary smart environments. Because modern IoT implementations overwhelmingly enforce robust encryption standards, such as TLS and DTLS (NISTIR 8259A), the payload contents are mathematically inaccessible to the investigator. Consequently, this raises the following problematic situation: investigators can successfully capture the encrypted network traffic, but they lack a standardized, step-by-step methodology to translate the remaining flow metadata—such as packet lengths, inter-arrival times, and throughput volumes—back into a reliable, chronological timeline of physical device events. Bridging this specific gap, without relying on payload decryption or physical device access, forms the primary objective of this research.

2.6.3 Conclusion and Justification of Proposed Methodology

The theoretical state of the art review presented in this subsection demonstrates a clear deficit in modern digital forensics. The inherent volatility of IoT hardware renders physical extraction unreliable, while the adoption of encryption neutralizes traditional network payload analysis. Although theoretical guidelines vestablish how devices *should* behave, investigators lack a practical, passive methodology to definitively prove how they *did* behave during a cyberattack.

To bridge this semantic gap, the subsequent chapters of this project will introduce, develop, and validate a novel methodology for IoT behavioral reconstruction. By strictly utilizing passive feature extraction on encrypted network traffic within a simulated smart home environment, this research

aims to provide investigators with a reliable toolset for generating chronological forensic timelines without requiring device-level access or payload decryption.

3 Methodology and Setup

3.1 Introduction

The theoretical research presented in the previous chapter established that traditional, device-centric digital forensics are largely ineffective in modern Internet of Things (IoT) ecosystems. Driven by extreme data volatility and the pervasive adoption of encrypted communications, investigators basically rely on network-centric methodologies. Consequently, this chapter has the objective of proving those theoretical concepts by proposing and testing a practical, passive network forensic methodology based on traffic feature extraction and behavioral analysis.

The primary objective of this experimental phase is to validate whether physical device states and malicious activities can be reliably inferred solely from encrypted network metadata. To achieve this in a controlled and academic manner, the experiment is conducted within a simulated smart home network. Due to budgeting constraints, the experiment was carried out by utilizing a simulated environment rather than physical hardware. This also ensures absolute control over the network topology, eliminates unexpected and undocumented background noise, and allows for the precise isolation of forensic artifacts. Furthermore, it guarantees that all experimental scenarios are strictly **replicable**.

3.2 Simulated Network Topology

To maintain consistency and ensure that any detected network anomalies are strictly the result of the simulated attacks, a standardized smart home topology was utilized across all experimental scenarios. The virtual network, orchestrated via Mininet, consists of a centralized gateway and three distinct end-nodes:

- **The Gateway (Open vSwitch):** Acts as the central routing hub for the smart home. All network traffic between devices, and between the devices and the external internet, flows through this node. It serves as the primary chokepoint where `tcpdump` is deployed to capture the experimental PCAPs.
- **Smart Camera Node:** A high-bandwidth virtual IoT device configured to simulate continuous or burst-driven TCP/UDP traffic, representing video streaming and periodic cloud check-ins.
- **Smart Thermostat Node:** A low-bandwidth virtual IoT sensor designed to generate periodic, highly predictable MQTT or HTTP "keep-alive" telemetry data. Due to the architectural nature of Open vSwitch in the Mininet environment, the gateway operates strictly at Layer 2 and lacks an assigned IP address. To facilitate communication and network validation, the Thermostat (10.0.0.2) is utilized as the logical gateway. It serves as the MQTT broker/NVR proxy for IoT traffic and the destination for ICMP keep-alive pings, ensuring the simulation remains functional despite the lack of a traditional Layer 3 gateway interface.
- **Attacker Node:** An external virtual machine introduced into the network infrastructure to execute the designated threat scenarios against the localized IoT devices.

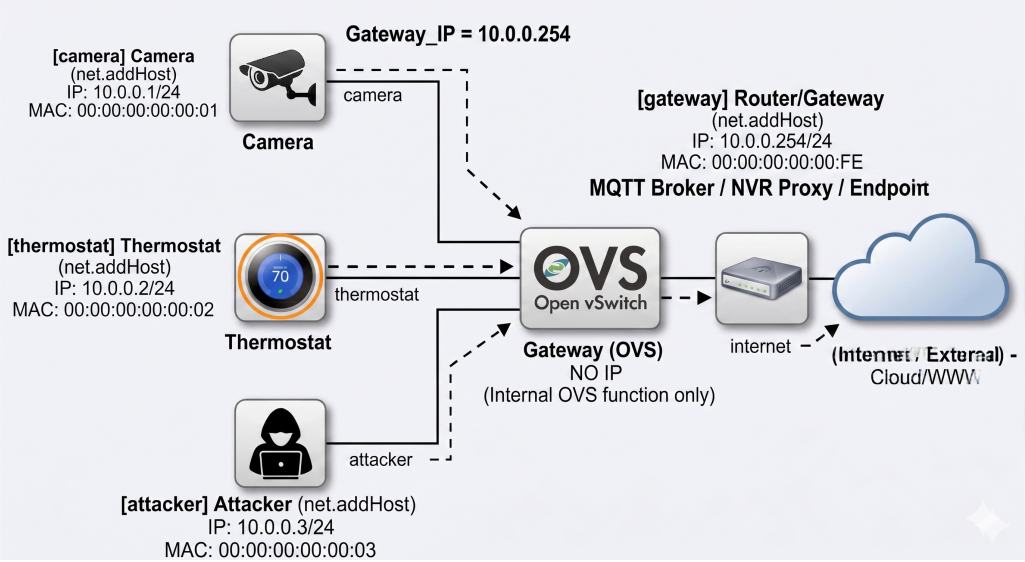

Figure 7: Home IoT Network Design

3.2.1 Baseline Behavioural Profile (MUD)

To establish the ground truth behaviour for each involved network node and provide a metric for anomaly detection in the attack scenarios, a strict Manufacturer Usage Description (MUD) profile was defined for both virtual IoT edge devices. These profiles dictate the exact network access control rules the devices must obey during normal operation.

Any traffic generated by these devices that deviates from the following allowed parameters is automatically classified as a forensic anomaly.

Smart Camera MUD Profile The virtual Smart Camera is modeled as a high-bandwidth device that requires time synchronization, DNS resolution, secure cloud command-and-control, and outbound UDP video streaming. Its approved network behaviors are strictly limited to the flows detailed in Table 1.

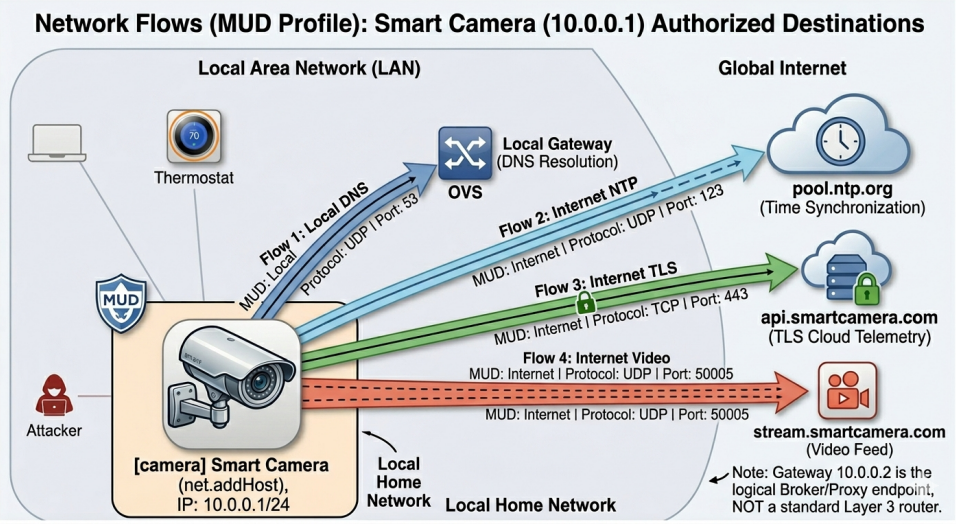

Figure 8: Smart Camera MUD Profile.

Direction	Protocol	Port	Authorized Destination
Local	UDP	53	Local Gateway (DNS Resolution)
Internet	UDP	123	`pool.ntp.org` (Time Synchronization)
Internet	TCP	443	`api.smartcamera.com` (TLS Cloud Telemetry)
Internet	UDP	50005	`stream.smartcamera.com` (Video Feed)

Table 1: Authorized Network Flows (MUD Profile) for the Smart Camera

Smart Thermostat MUD Profile The virtual Smart Thermostat is modeled as a low-bandwidth and repetitive behaviour sensor. It does not constantly transmit data, it relies on lightweight and periodic *keep-alive* messages utilizing secure MQTT (Message Queuing Telemetry Transport) as well as reporting the temperature every 6 minutes, 10 times per hour. Its approved behaviors are detailed in Table 2.

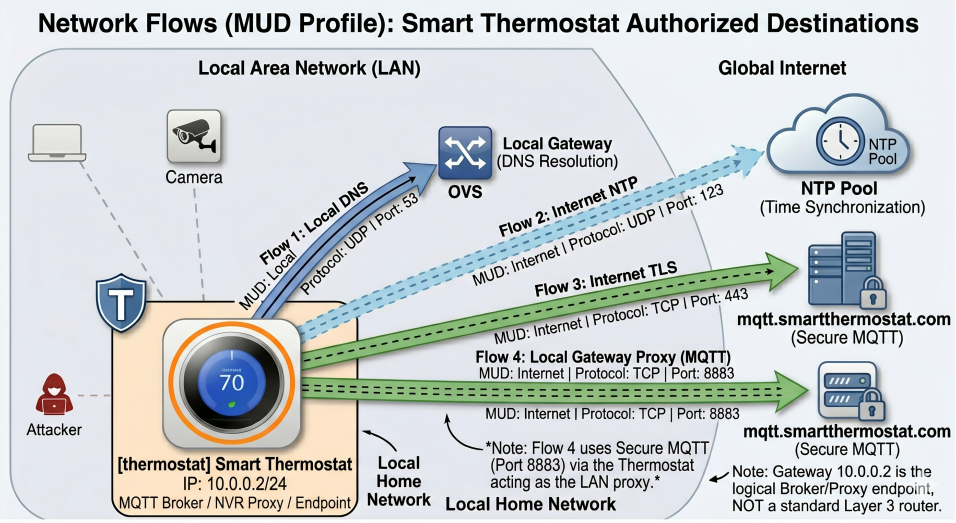

Figure 9

Direction	Protocol	Port	Authorized Destination
Local	UDP	53	Local Gateway (DNS Resolution)
Internet	UDP	123	`pool.ntp.org` (Time Synchronization)
Internet	TCP	8883	`mqtt.smartthermostat.com` (Secure MQTT)

Table 2: Authorized Network Flows (MUD Profile) for the Smart Thermostat

3.3 Tools

To conduct this research, a specialized, open-source forensic toolchain was deployed within an isolated Ubuntu 22.04 Linux virtual environment:

- **Network Emulation (Mininet [25]):** Utilized to construct the Software-Defined Networking (SDN) topology, acting as the virtual smart home gateway and generating synthetic IoT edge nodes.
- **Traffic Acquisition (Wireshark [26]/tcpdump [27]):** Deployed at the virtual gateway to act as an immutable observer, capturing all passing communications and generating raw Packet Capture (PCAP) files.

- **Feature Extraction (Zeek [28]):** Functioning as the core network security monitor, Zeek was employed to parse the encrypted PCAPs and extract readable, structured metadata logs in order to not rely on Deep Packet Inspection (DPI).
- **Data Analysis (Python):** Zeek logs were subsequently ingested into Python (utilizing the Pandas [29] and Matplotlib libraries) to filter background noise and visualize the behavioral fingerprints of the devices.

3.4 Experimental Scenarios

To thoroughly evaluate the effectiveness of the proposed feature extraction methodology, the experimental phase is divided into four distinct scenarios. Each scenario is designed to leave a uniquely identifiable "fingerprint" within the network metadata.

1. **Scenario 1: Operational Baseline (Normal Behavior):** In this control scenario, no malicious activity occurs. The IoT devices boot, request IP addresses, synchronize time via NTP, and transmit their standard telemetry. This capture establishes the "ground truth" baseline for Inter-Arrival Times (IAT) and standard flow volumes, acting as the reference point for all subsequent anomaly detection. This scenario is the most important since it is useful for establishing the strict reference point required for all subsequent anomaly detection scenarios.
2. **Scenario 2: Network Reconnaissance and Brute-Force Authentication:** The external attacker node performs a rapid port scan to map the smart home, followed by a concentrated password brute-force attack against the Smart Camera. This scenario tests the methodology's ability to detect localized, high-frequency connection rejections. The main forensic objective of this scenario is the isolation of the metadata artefacts produced by the unauthorized access attempts, specifically the dense cluster of TCP flows generated by the Network Reconnaissance.
3. **Scenario 3: Unauthorized State Change and Data Exfiltration:** Following a simulated compromise, the attacker triggers the Smart Camera to begin streaming high-density data to an unauthorized external IP address. This scenario evaluates the capacity to map a sudden spike in volumetric data and altered packet lengths to a specific physical state change (camera activation). To validate the existence of a breach of confidentiality, statistical deviations in the outbound packet lengths and longer flow durations will be need to be observed, even when destination IP attempts to masquerade as benign.
4. **Scenario 4: Botnet Infection (Outbound DDoS):** Simulating a Mirai-style malware infection, the low-bandwidth Smart Thermostat is hijacked and instructed to flood an external web server with outbound TCP SYN packets. This scenario demonstrates how to identify severe violations of a device's expected Manufacturer Usage Description (MUD) profile and baseline whitelist by actions such as unauthorized transport protocols use or external IP communications that can indicate a loss of device functionality integrity.

3.5 Forensic Analysis Pipeline

To ensure the validity, reliability, and academic rigour of the experimental results, the data generated in each scenario is subjected to a standardized and repeatable forensic analysis pipeline. This methodology is based on standard digital forensic principles, specifically aligning with guidelines for network forensic techniques and incident response (such as NIST SP 800-86).

The following four-phased pipeline was designed to reconstruct device behaviour and identify malicious anomalies utilizing only network metadata and statistical feature extraction, since the existing ineffectiveness of DPI due to the use of encrypted communications in the simulated IoT environment. The followed forensic analysis pipeline has been divided into four distinct phases:

1. **Phase 1: Evidence Integrity and Preservation (Collection):** In digital forensics, proving the integrity of the original evidence is paramount. Immediately following the conclusion of each Mininet simulation, a cryptographic hash (SHA-256) of the resulting raw `.pcap` file is computed and recorded. Subsequently, exact working copies of these datasets are generated for the actual analysis, ensuring the original files remain strictly immutable. This combined process of cryptographic hashing and evidence duplication mathematically guarantees that the core network captures have not been tampered with or accidentally altered prior to the examination phase, thereby establishing a reliable chain of custody for the digital artifacts.
2. **Phase 2: Network Feature Extraction (Examination):** The hashed `.pcap` files are subsequently processed through the Zeek Network Security Monitor. Zeek acts as the primary examination engine, passively translating the raw binary network traffic into structured, human-readable metadata logs without attempting to decrypt the payloads. For the scope of this research, the extraction focuses heavily on the `conn.log` (which records connection states, transport protocols, transported byte volumes, and flow durations) and the `dns.log` (which records plaintext domain name resolutions).
3. **Phase 3: Data Normalization and Filtering (Analysis - Part I):** The extracted Zeek metadata is then ingested into a Python data analysis environment utilizing the Pandas library. In this phase, the raw datasets are cleansed of environmental background noise, such as standard Mininet overhead and Layer 2 broadcast traffic. The dataset is filtered to isolate the flows specifically associated with the target IoT IP addresses and the gateway. Finally, critical behavioral metrics are mathematically derived from the timestamps, most notably the Inter-Arrival Time (IAT) between specific device communications.
4. **Phase 4: Behavioral Reconstruction and Anomaly Detection (Analysis - Part II):** The final phase involves the forensic deduction and timeline reconstruction. For Scenario 1, the normalized metrics are evaluated to establish the statistical mean and variance of the devices' normal operational states ("Ground Truth"). For Scenarios 2, 3, and 4, the extracted features are cross-examined against this established baseline. By identifying statistically significant deviations—such as the dense temporal clustering of rejected connections, volumetric spikes in outbound data, or unauthorized protocol utilization—the exact timestamp of the network compromise is identified, allowing for a chronological reconstruction of the attacker's actions.

The four phases are better represented in this diagram:

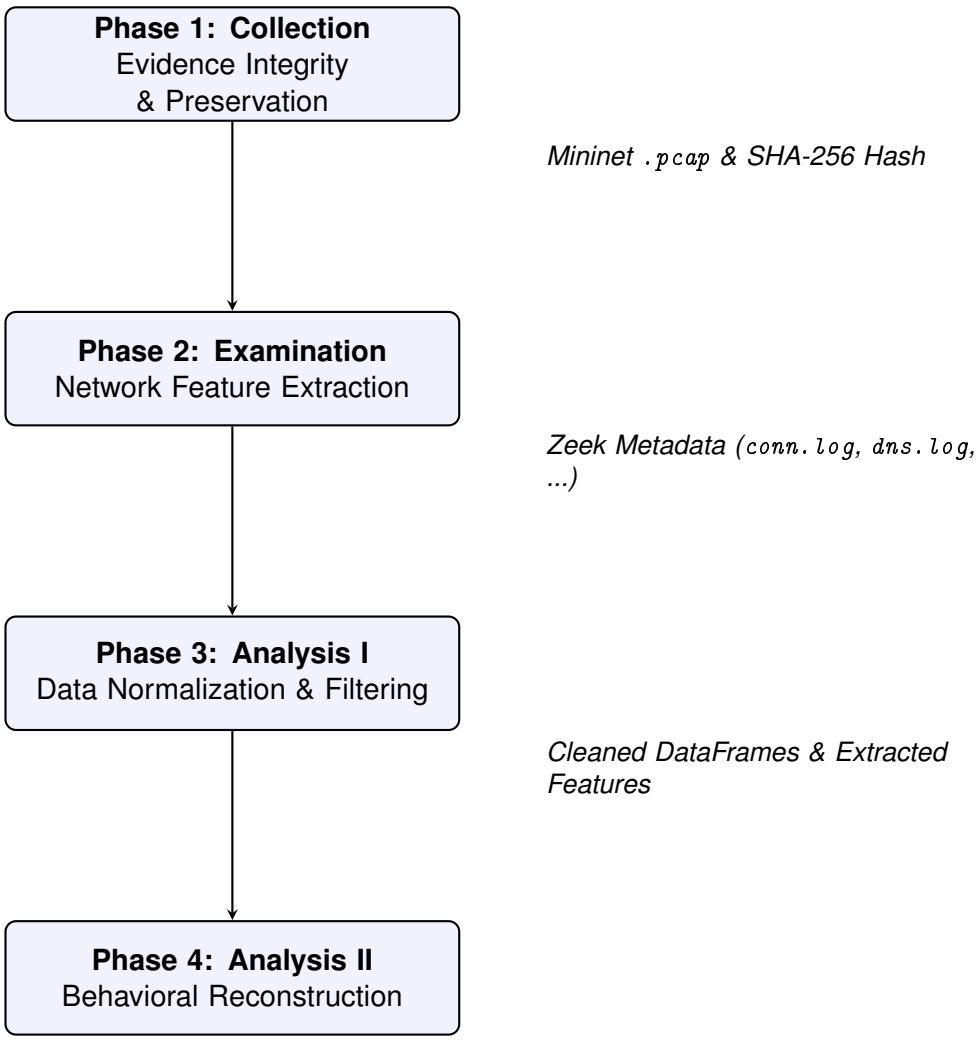

Figure 10: The standardized 4-phase network forensic analysis pipeline applied to all experimental scenarios.

4 Experimental Execution

With the smart home environment topology established and the foresinc toolchain configured in the forensic experimental workstation, this section details the practical execution of the four predefined scenarios. The primary aim is to generate, capture and analyze the network traffic associated with both normal IoT operations and the specific selected cyber threat vectors. For each scenario, the methodology follows a strict procedural pipeline:

1. Orchestrate Mininet network simulation.
2. Acquire raw packet capture at the gateway
3. Extract metadata using Zeek
4. Perform a behavioural analysis of the resulting flows

By comparing the anomalous traffic patterns generated in Scenarios 2, 3, and 4 against the operational baseline established in Scenario 1, this research validates the efficacy of passive network feature extraction for forensic timeline reconstruction.

4.1 Simulation Setup and Execution

To execute the scenarios, a Python script template sctructure has been defined. Utilizing the Mininet API, the scripts are deployed to orchestrate the virtual devices. The scripts instantiate the Open vSwitch gateway and configure the host nodes to generate specific traffic patterns mimicking physical IoT hardware. The template is modified to simulate the objectives defined in each scenario.

For the **Smart Thermostat**, the simulation script utilizes simple scheduling commands (`cron` or Python `time.sleep` loops) to generate periodic TCP traffic directed at port 8883, simulating a secure MQTT "keep-alive" heartbeat every 10 seconds as well as periodical temperature measurements every 10 minutes. Furthermore, periodic UDP requests to port 123 were generated to simulate standard NTP time synchronization.

For the **Smart Camera**, a continuous stream of background UDP traffic was generated using the `iperf` utility, to accurately simulate a live, outbound video feed. Additionally, synthetic TLS handshakes were periodically executed against TCP port 443 to represent secure cloud telemetry check-ins. The smart camera also performs periodic UDP requests to port 123 to simulate standard NTP time synchronization.

Following the conclusion of the Mininet simulation, the resulting PCAP file of each scenario was processed using Zeek. Because the simulated payloads are encrypted, Deep Packet Inspection (DPI) has been discarded. Instead, Zeek successfully parsed the capture into structured metadata logs, specifically generating a `conn.log` (detailing connection states, bytes transferred, and durations) and a `dns.log` (documenting the plaintext domain resolution requests).

Finally, all the extracted metadata from all experimental runs underwent a comparative behavioural analysis. By comparing network features such as Inter Arrival Times (IAT), flow durations, and volumetric byte transfers with the baseline fingerprints of the devices. The comparison of the attack based metrics together with the baseline allows drawing the definitive conclusions regarding the viability of passive IoT network forensics.

4.1.1 Chain of Custody

Before summarizing the different scenario results, the hashes of each used evidence are collected in this table as a proof of chain of custody.

Experimental Phase	Evidence File (PCAP)	SHA-256 Cryptographic Hash	Integrity
Scenario 1: Baseline	originals/scenario1_baseline.pcap	d6bf23c441bb69c46f89c87bb7e2f457baf4cac4f910372332722b228bd12936	✓ Verified
	working_copies/scenario1_working_copy.pcap	d6bf23c441bb69c46f89c87bb7e2f457baf4cac4f910372332722b228bd12936	
Scenario 2: Recon & Brute-Force	originals/scenario2_recon_bruteforce.pcap	a94fcc7c7ef145fcdbb1bc4f70134005703ca5cd38b3b440969f67ea03abd094	✓ Verified
	working_copies/scenario2_working_copy.pcap	a94fcc7c7ef145fcdbb1bc4f70134005703ca5cd38b3b440969f67ea03abd094	
Scenario 3: Exfiltration	originals/scenario3_exfiltration.pcap	c3fd72fa20c38d241cb089c866a5e27a24fe264f0a1d04a208de53c3d35b0c17	✓ Verified
	working_copies/scenario3_working_copy.pcap	c3fd72fa20c38d241cb089c866a5e27a24fe264f0a1d04a208de53c3d35b0c17	
Scenario 4: Botnet DDoS	originals/scenario4_botnet.pcap	0be2d975155a51dbbab00fb96301bb66d30fed5138694c5abfd7d6c3dec9de70	✓ Verified
	working_copies/scenario4_working_copy.pcap	0be2d975155a51dbbab00fb96301bb66d30fed5138694c5abfd7d6c3dec9de70	
Zip files	original_scenarios.tar.xz	fb0fe531f28d635dd614bd2bb914f4ec87845acb55f6507be17b43f340caafb4	✓ Verified
	working_scenarios.tar.xz	7cd22d49fb7e893ee4865d4e6a0b39728541b884ea759dc17f4a7b0b0a86ecc5	

Table 3: Cryptographic Chain of Custody: SHA-256 hash verification confirming zero data degradation between the original Mininet captures and the working copies analyzed via Zeek.

4.2 Scenario 1: Operational Baseline Behaviour

4.2.1 Introduction and Objectives

Scenario 1 serves as the control environment for the entirety of the experimental phase. In this scenario, the smart home network operates under standard, benign conditions without any external interference or malicious activity from the attacker node. The primary objective is to establish a verifiable "ground truth" of the network's behavior. The scenario goals in detail are the following:

1. Generate realistic, benign background traffic that strictly adheres to the Manufacturer Usage Description (MUD) profiles defined previously in Table 1 (Smart Camera) and Table 2 (Smart Thermostat).
2. Capture the raw network interactions during standard operational states such as device data transmission, time synchronization, and periodic telemetry reporting.
3. Extract and catalogue baseline network metadata. This includes metrics such as flow durations, expected packet lengths, and Inter-Arrival Times (IAT)—to, with the objective to use them as the definitive reference point for anomaly detection in subsequent attack scenarios.

4.2.2 Expected Behaviour and Theoretical Baselines

To ensure statistical significance and provide a robust dataset for feature extraction, the Scenario 1 baseline simulation was parameterized for a continuous 1200 seconds (20-minute) capture period. Operating under ideal, interference-free conditions, the expected behavioural fingerprint for the network topology is highly predictable. **To accurately calculate the total expected flows, we must account for inclusive boundary timing. Because the communication loop initializes at exactly $t = 0$ and concludes at $t = 1200$, both the initial and final packets are captured, necessitating an $n + 1$ adjustment to the final count.**

- **Smart Thermostat (Deterministic Telemetry):** The device is programmed to generate a rigid temporal pulse of MQTT "keep-alive" TCP flows directed at port 8883 at precise 10-second intervals. Over the 1200-second capture window, this establishes a target Inter-Arrival Time (IAT) of exactly 10 seconds and yields 121 deterministic pulses (accounting for inclusive boundary timing). This baseline is punctuated by heavier TCP payloads representing environmental telemetry updates at 300-second intervals, producing exactly 5 discrete transmissions.
- **Smart Camera (Volumetric Streaming):** The capture is expected to demonstrate a continuous, high-density stream of outbound UDP traffic on port 50005, simulating a live multimedia feed. This volumetric plateau must maintain an empirical target bandwidth of approximately 500 KB/s. The continuous stream is structurally punctuated by low-frequency TCP connections on port 443, representing synthetic TLS cloud check-ins. Firing at 300-second intervals, these telemetry data establish an expected baseline of 5 distinct TLS sessions.
- **Network Services (Time Synchronization):** To maintain cryptographic and operational clock accuracy, the combined IoT environment generates periodic Network Time Protocol (NTP) synchronization requests (UDP port 123) directed to the local gateway. With a polling interval of 600 seconds, the baseline dictates exactly 3 requests per device, resulting in an aggregate environmental footprint of 6 NTP queries.
- **Domain Name Resolution (DNS):** Both edge devices rely on the local gateway (UDP port 53) to resolve their respective cloud endpoints. Enforcing a strict no-caching policy to maximize experimental visibility, the Smart Camera and Smart Thermostat independently execute DNS queries every 60 seconds. This overlapping operational frequency produces a highly predictable application-layer baseline of 42 total queries (21 per device).

- **Attacker Node (Zero-Footprint Baseline):** Because Scenario 1 establishes the benign operational control group, the designated adversary node must remain perfectly dormant. The theoretical baseline demands an absolute mathematical void: exactly zero network flows originating from or destined to the attacker's IP address (`10.0.0.3`), ensuring a scientifically pristine experimental environment.

4.2.3 Evidence Integrity and Feature Extraction

In strict adherence to the defined forensic methodology (Phase 1), the integrity of the original network capture must be cryptographically verified before any analysis occurs. Following the 20 minutes baseline simulation, the raw traffic was exported as `scenario1_baseline.pcap`.

To establish a mathematically verifiable chain of custody, a SHA-256 hash of the original evidence was computed. Subsequently, an immutable working copy was generated, and its hash was explicitly verified against the original to ensure absolute data fidelity during the duplication process. This procedure is documented in the terminal execution below:

```
thismanera@thismanera:~/Documents/cfca/iot_forensics_project$ sha256sum pcaps/originals/scenario1_baseline.pcap
d6bf23c441bb69c46f89c87bb7e2f457baf4cac4f910372332722b228bd12936  pcaps/originals/scenario1_baseline.pcap
thismanera@thismanera:~/Documents/cfca/iot_forensics_project$ cp pcaps/originals/scenario1_baseline.pcap pcaps/working_copies/scena
rio1_working_copy.pcap
thismanera@thismanera:~/Documents/cfca/iot_forensics_project$ sha256sum pcaps/working_copies/scenario1_working_copy.pcap
d6bf23c441bb69c46f89c87bb7e2f457baf4cac4f910372332722b228bd12936  pcaps/working_copies/scenario1_working_copy.pcap
```

Figure 11: Scenario 1: Terminal execution demonstrating the SHA-256 hashing and working copy verification.

4.2.4 Baseline Validation and Extracted Metrics

Following the successful verification of the working copy, the original evidence file was securely isolated to prevent accidental modification. The duplicated working copy was then processed through the Zeek Network Security Monitor (Phase 2).

Utilizing the `-C` directive to bypass virtualized checksum offloading errors inherent to the Mininet environment, Zeek successfully parsed the raw binary traffic into structured metadata logs. The resulting artifacts, predominantly the `conn.log` and `dns.log`, were exported to a dedicated directory, yielding a clean, noise-reduced dataset ready for ingestion and behavioral analysis.

```
thismanera@thismanera:~/Documents/cfca/iot_forensics_project$ ls zeek_logs/scenario1_logs/
thismanera@thismanera:~/Documents/cfca/iot_forensics_project$ zeek -C -r pcaps/working_copies/scenario1_working_copy.pcap Log::defa
ult_logdir=zeek_logs/scenario1_logs
thismanera@thismanera:~/Documents/cfca/iot_forensics_project$ ls zeek_logs/scenario1_logs/
conn.log  dns.log  ntp.log  packet_filter.log
```

Figure 12: Scenario 1: Zeek log extraction of Scenario 1.

The resulting Zeek logs were ingested into a Python-based Jupyter Notebook to facilitate the behavioral analysis of the simulation.

MUD Profile Validation

The primary objective of the baseline scenario analysis is to verify that the simulated IoT devices adhered strictly to their programmed MUD profiles during the simulation execution. By cross-referencing the theoretical behavioral parameters against the empirical Zeek metadata, the operational integrity of the smart home topology can be definitively proven.

The following table compares the expected flows versus the empirical flows. As it can be observed, since it is the baseline scenario, a 100% accuracy is achieved.

Traffic Classification Profile	Expected Flows	Empirical Flows	Status / Delta
Infrastructure & Core Services			
DNS Queries (UDP/53)	42	42	✓ Exact Match
NTP Synchronization (UDP/123)	6	6	✓ Exact Match
Smart Thermostat (10.0.0.2)			
MQTT Keep-Alives (TCP/8883)	121	121	✓ Exact Match
MQTT Telemetry Payloads (TCP/8883)	5	5	✓ Exact Match
Smart Camera (10.0.0.1)			
Video Feed Volumetric Stream (UDP/50005)	1	1	✓ Exact Match
TLS Cloud Telemetry (TCP/443)	5	5	✓ Exact Match
Environmental Noise / Unaccounted			
Background ICMP (Gateway → Camera)	0	1	∼ Minor Hypervisor Noise
Total Captured Flows	**180**	**181**	**100% Accuracy**

Table 4: Scenario 1: Theoretical Expected Traffic vs. Empirical Zeek Capture Baseline

As expected, no attacker activity has been registered.

Camera Streaming Bandwidth

For visualization purposes, the network bandwidth was recalculated from bits per second to Kilobytes per minute (KB/min). Given a target bandwidth of 500 KB/s (configured as 4,000 Kbps in the pcap_generator code), the expected bandwidth is 29,296.88 KB/min. The empirical results demonstrated an average of 27,857.80 KB/min. As seen in the figure below, this variance is entirely attributed to the brief temporal delays required to establish maximum transmission speed at $t=0$, as well as the socket teardown phase prior to the 1200-second mark. Excluding these boundary periods, the sustained bandwidth strictly adheres to the expected operational parameters.

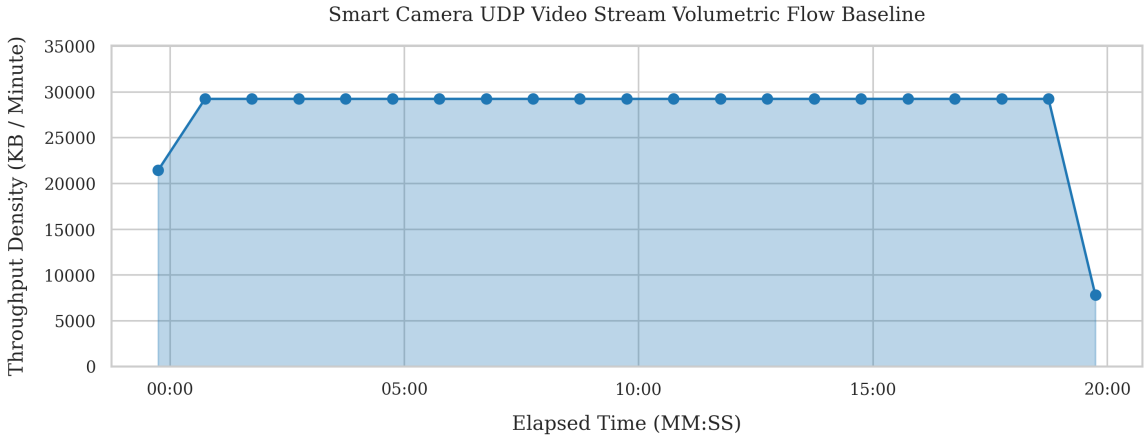

Figure 13: Scenario 1: Smart Camera Bandwidth evolution over experiment time.

In addition, the camera directional byte ratio has been calculated. From all the traffic exchanged between the camera and the gateway, 99.9998% of the traffic has been sent by camera, while the rest are responses from the gateway. This is normal since the camera must act as data flow transmitter.

Thermostat IAT Keep Alives

The temporal delta between consecutive keep-alive messages was calculated to assess baseline

stability. While the theoretical expected interval is strictly defined at 10.0 seconds, the empirical results demonstrated a near-perfect mean Inter-Arrival Time (IAT) of 10.0001 seconds. Furthermore, the calculated standard deviation of just 0.000449 seconds proves that the execution overhead of the virtualized environment introduces negligible variance to the device's automated reporting loop.

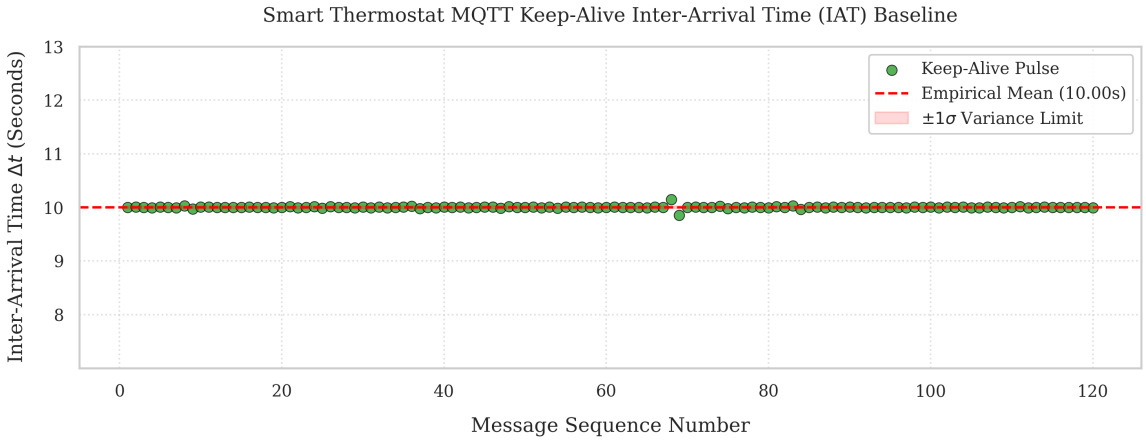

Figure 14: Scenario 1: IAT of Smart Thermostat Keep Alive messages.

In addition, the mean size of the temperature reports generated by the Smart Thermostate has been calculated. The standard size is of 27 bytes. This metric will be important in Scenario 4 since the Thermostat is expected to send multiple packets with a bigger size to simulate it has been infected as a botnet.

Other Metrics

A interaction heatmap has been developed to track and validate the expected communication patterns between the network's internal nodes. This matrix serves as visual proof of the environment's operational baseline, mapping which cross-node communications are authorized by the Manufacturer Usage Description (MUD) profiles. It should be noted that traffic originating from the gateway (inbound responses) was excluded from this visualization. This methodological scoping decision was made to filter out infrastructure noise, focusing the analytical lens strictly on the behaviors, vulnerabilities, and potential lateral movements of the IoT devices themselves during the attack simulations.

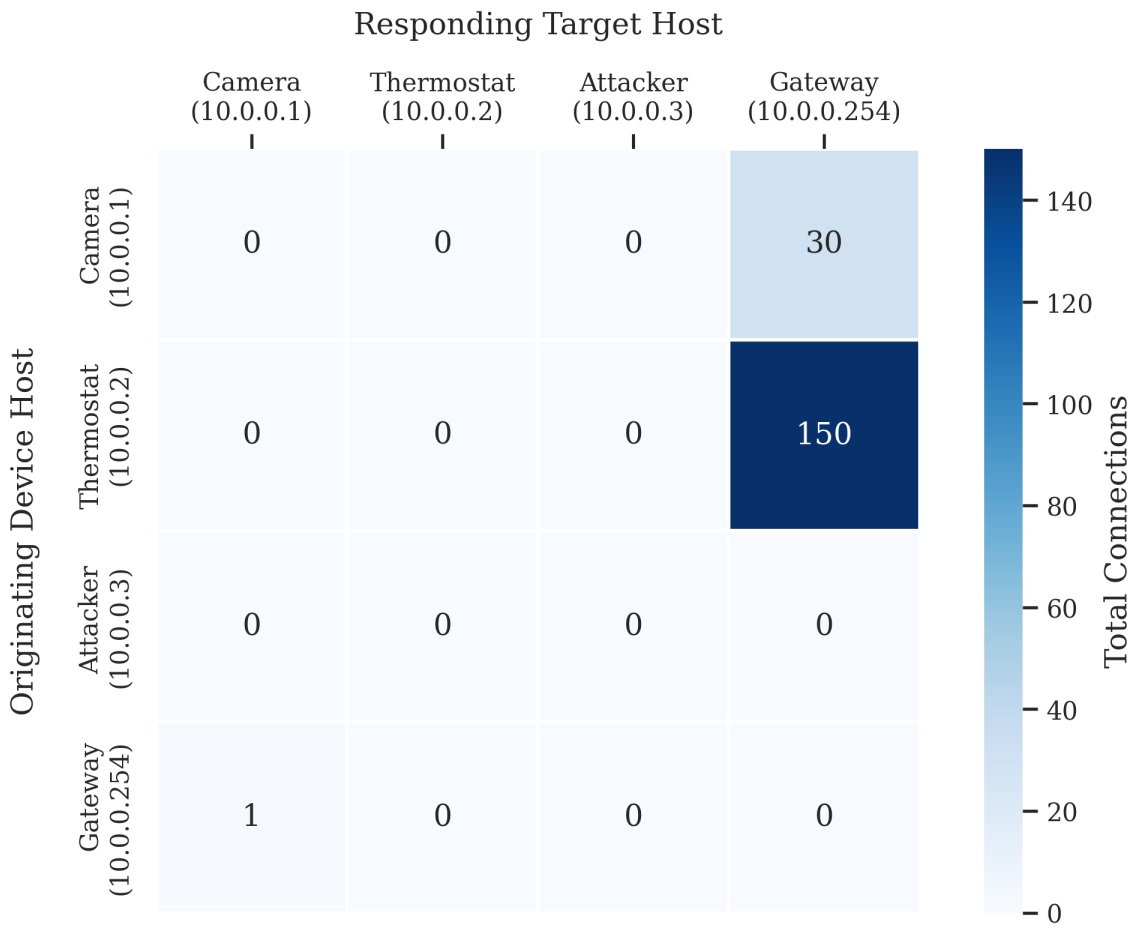

Figure 15: Scenario 1: Empirical interaction heatmap detailing authorized internal network flows.

Finally, another interesting metric is to track the TCP connection state distribution. In a secure, uncompromised IoT network, TCP sessions must exhibit highly disciplined behavioral patterns. Legitimate devices connect to designated cloud endpoints, execute their application-layer handshakes, and terminate gracefully. Therefore, this baseline metric should be heavily dominated by SF (Normal connection establishment and termination) since all the network traffic flows have been programmed to start and end in the duration of the experiment.

As it can be observed all the connections are correctly finalized as expected.

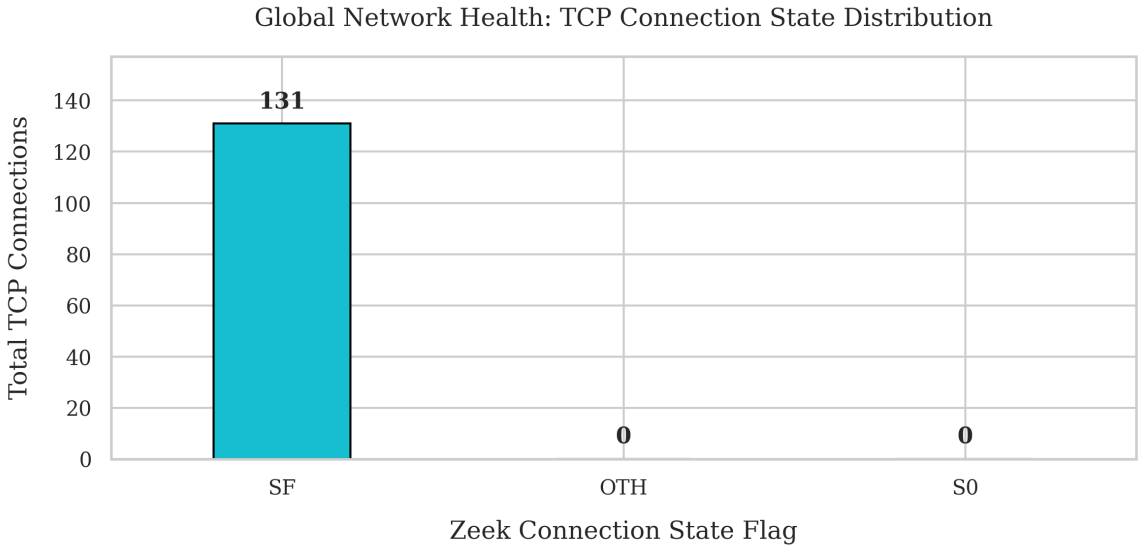

Figure 16: Scenario 1: TCP Connection State Distribution.

4.3 Scenario 2: Network Reconnaissance and Brute-Force Authentication

4.3.1 Introduction and Objectives

Scenario 2 introduces the first active adversarial engagement within the experimental smart home network. In this phase, the previously established zero-trust architecture is subjected to targeted malicious activity originating from the internal attacker node (`10.0.0.3`), simulating a compromised lateral asset. The attacker has not been designed as an external entity, since this would complicate the architectural setup without adding an extra valuable forensic value. The primary objective is to measure and document the network's structural and statistical deviations from the Scenario 1 control baseline. The scenario goals in detail are the following:

1. Execute internal network reconnaissance and targeted port scanning against the IoT edge devices. This phase captures the systemic footprint of unauthorized topology mapping, specifically documenting the resulting surge in aborted and rejected TCP connection states.
2. Launch a volumetric credential brute-force attack targeting the Smart Camera's local TLS control channel (TCP port 443), capturing the exponential spike in network flow density as the attacker attempts to guess the administrative password.
3. Extract and catalogue the resulting adversarial network metadata. By comparing metrics such as TCP state distributions, node-to-node adjacency matrices, and directional byte ratios against the Scenario 1 ground truth, the objective is to definitively prove the violation of the authorized MUD profiles.

4.3.2 Expected Behaviour and Theoretical Baselines

To ensure statistical significance and provide a robust dataset for feature extraction, the Scenario 2 has also been executed for 1200 seconds (20-minutes) capture period.

This scenario expects to obtain the same behaviour from the network with the addition of new ones:

- **Lateral Reconnaissance (TCP State Disruption):** The attacker node will execute a subnet scan. This will manifest forensically as a massive volumetric surge in `REJ` (Rejected) and `S0` (Attempted, no reply) Zeek connection states as the scanner enumerates closed ports, drastically skewing the healthy `SF` baseline established in Scenario 1. Consequently the number of

connection flows together will increase, specially generating a huge spike during the network scan.

- **High-Density Connection Spikes (Credential Brute-Force):** The targeted attack on the Smart Camera's local TLS control channel (TCP port 443) will shatter the flat, low-density operational baseline. We expect to capture a constant spike in the Network Flow Density (flows per minute), representing rapid, iterative password guessing from the adversary.
- **Adjacency Matrix Violation (MUD Corruption):** The strict node-to-node isolation observed in the control scenario will be visibly broken. The Topology Interaction Heatmap will log unauthorized lateral traffic originating from `10.0.0.3` and terminating at all the devices of the network (network scan), as well as an increased number of packets for the Camera (`10.0.0.1`) due to the brute force attack.

4.3.3 Evidence Integrity and Feature Extraction

The same steps of 4.2.3 have been followed to guarantee the integrity and validity of the extracted features.

MUD Profile Validation

The first step is to compare the expected behaviour seen in the previous scenario with the empirical results to detect attacker patterns.

Traffic Classification Profile	Expected	Empirical	Status / Delta
Authorized IoT Services			
DNS Queries (UDP/53)	42	42	✓ Baseline Match
NTP Synchronization (UDP/123)	6	6	✓ Baseline Match
MQTT Keep-Alives (TCP/8883)	121	121	✓ Baseline Match
MQTT Telemetry Payloads (TCP/8883)	5	5	✓ Baseline Match
Video Feed Stream (UDP/50005)	1	1	✓ Baseline Match
TLS Cloud Telemetry (TCP/443)	5	5	✓ Baseline Match
Adversarial Activity (10.0.0.3)			
Network Reconnaissance	0	3,074	× Attack Detected
Credential Brute-Force	0	2918	× Attack Detected
Total Connection Flows	**180**	**5,992**	**3,329% Volumetric Spike**

Table 5: Scenario 2: Comparative Traffic Breakdown (Authorized vs. Adversarial)

As expected, attacker activity has been registered. The empirical data is characterized by a massive volumetric surge, contributing the overwhelming majority of the 5,992 attacker-involved connections. Specially, because the scanner indiscriminately enumerates port ranges across the `10.0.0.0/24` architecture, causing an aggregate network density of a 3,329% volumetric spike.

This massive structural deviation proves that while legitimate device operations continue to execute in the background, they are effectively buried under a high-density noise floor during an active re-

connaissance campaign. This provides an undeniable, quantifiable signature for anomaly detection engines.

Camera Behaviour
The observed activity from the Smart Camera matches the MUD, since the attack does not target its bandwidth or other messages. Basically, it tries to perform a brute force attack.

Thermostat Behaviour
The Smart Thermostat sees no changes in behaviour since it is not the target of the attack.

Other Metrics

As illustrated in Figure 17, the empirical flow density exhibits three distinct volumetric anomalies. Rather than a random distribution of noise, these deviations perfectly map to the chronological execution of the adversarial kill chain:

- **Phase 1: The Reconnaissance Surge (Initial Spike):** The massive, immediate vertical spike observed at the beginning of the timeline represents the automated subnet and port sweep. Because network scanners operate asynchronously—flooding the target with thousands of simultaneous `SYN` probes to enumerate open ports—this phase generates an explosive but brief density maximum before the scanner concludes its mapping phase.
- **Phase 2: The Brute-Force Plateau (Sustained Volatility):** Following the initial scan, the graph does not return to the authorized baseline of $\sim$8 flows per minute. Instead, it transitions into a sustained plateau. This elevated constant peak represents the credential brute-force attack against the Smart Camera's TLS control channel (TCP port 443). The script executes rapid, iterative login attempts, generating a persistent high-density noise floor that persists for the majority of the attack window.

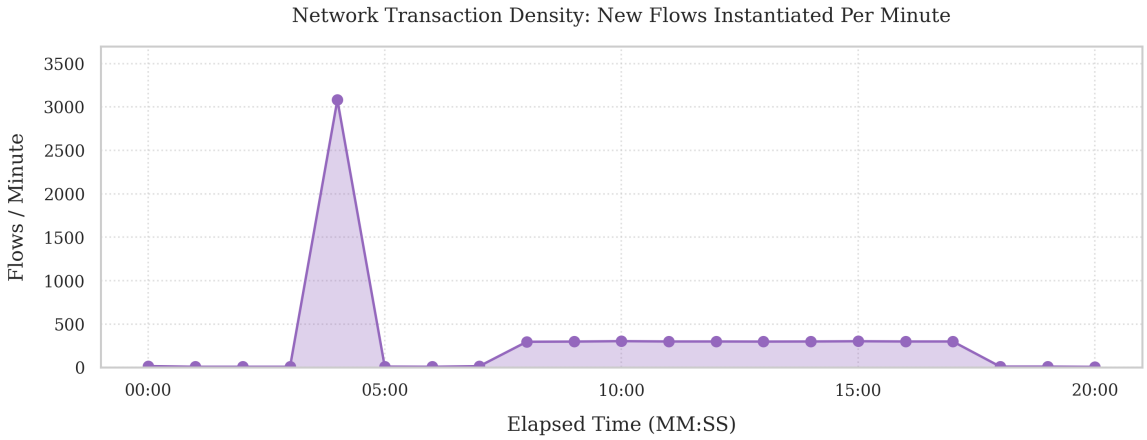

Figure 17: Scenario 2: Network Flow Density.

The TCP connection state distribution, also provides clear structural proof of the attack's impact on the network. As seen in Scenario 1, In a stable baseline, TCP flows primarily resolve with an `SF` (Normal) state flag, indicating successful connections and clean teardowns. While authorized IoT traffic continued normally during the attack (registering 133 `SF` flows), it was completely eclipsed by a massive surge in anomalous activity.

Specifically, Zeek recorded 5,987 flows terminating in a `REJ` (Rejected) state. This flag is logged when a target actively refuses a connection attempt via a TCP Reset (RST) packet. This spike in rejected handshakes is the direct footprint of the attacker (`10.0.0.3`) scanning the network. As the scanner flooded the IoT edge devices with `SYN` probes targeting closed ports, the devices continuously issued RST packets to block the unauthorized access.

Consequently, this sudden inversion of the `SF`-to-`REJ` ratio serves as an ideal, automated tripwire for detecting an active internal breach.

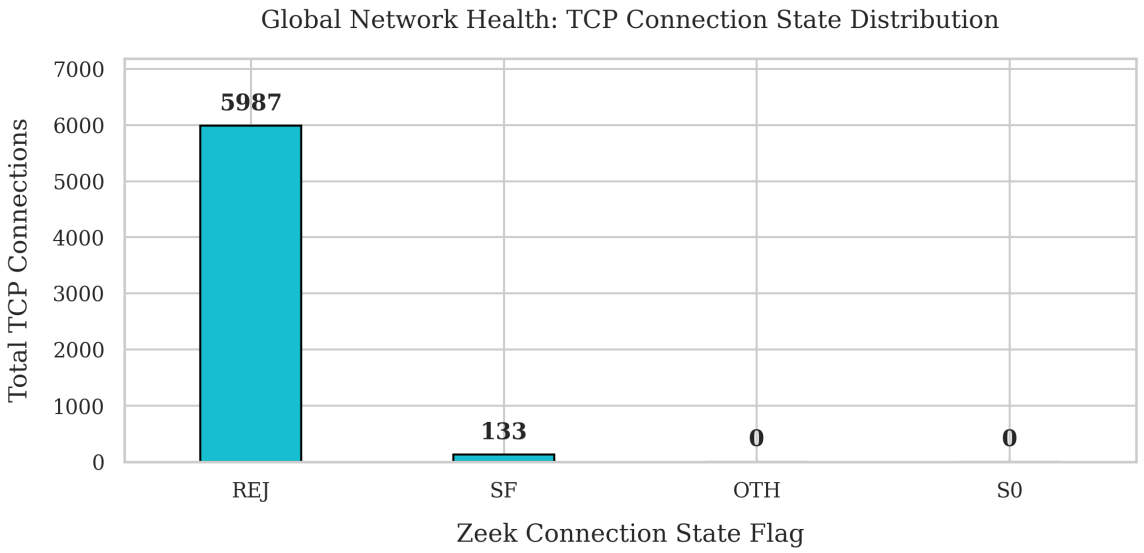

Figure 18: Scenario 2: TCP Connection State Distribution.

Finally, the interaction heatmaps is totally altered, in this scenario is exposing a network heavily saturated with unauthorized cross-node communications. The baseline has been broken by the attacker node which acts as the origin for all anomalous traffic. The matrix clearly distinguishes the broad network reconnaissance, evidenced by the 1,025 probes sent to both the Gateway and the Thermostat, from the targeted exploitation of the Smart Camera, which absorbed 3,942 direct connections.

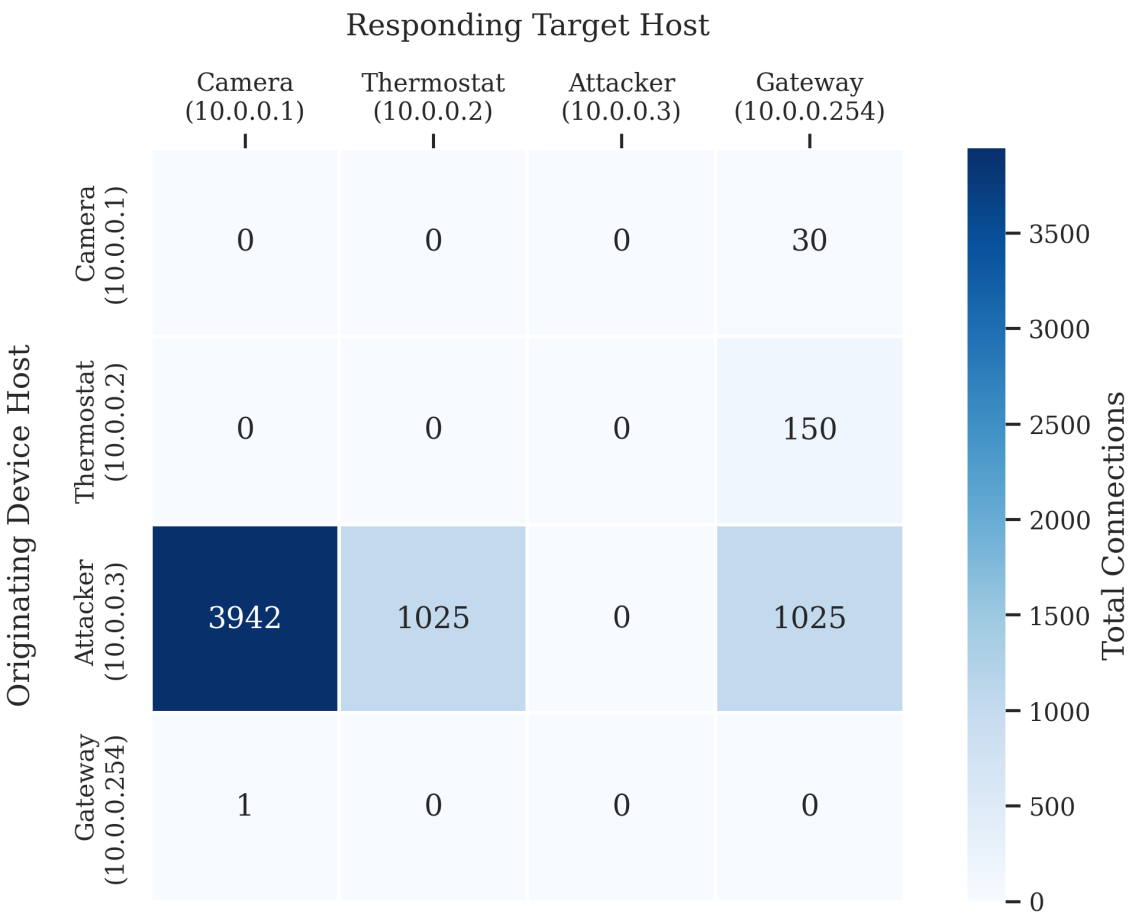

Figure 19: Scenario 2: Empirical interaction heatmap detailing authorized internal network flows.

4.4 Scenario 3: Unauthorized State Change and Data Exfiltration

4.4.1 Introduction and Objectives

Scenario 3 advances the adversarial narrative estblished in the previous scenario from initial network compromise to post-exploitation, focusing specifically on data exfiltration and stealthy Command and Control (C2) communication. In this phase, the previously compromised Smart Camera is manipulated by the internal attacker node (`10.0.0.3`) to completely bypass its authorized constraints. To capture this dynamic, a new external adversarial (`203.0.113.5`) is introduced to act as the destination for the stolen data. The primary objective is to measure and document how post-exploitation routing anomalies and application-layer manipulations deviate from the Scenario 1 control baseline. The scenario goals in detail are the following:

1. Execute a targeted trigger payload from the internal attacker to the Smart Camera's TLS control channel at a precise temporal marker ($t = 300$ seconds). This simulates the localized activation of malware or a malicious configuration update, initiating the exfiltration sequence without generating broad network noise.
2. Demonstrate rogue data exfiltration and port masquerading. The Smart Camera will be forced to terminate its authorized local video stream and redirect the high-bandwidth UDP payload directly to the external rogue IP address. This exfiltration stream is deliberately mapped to port

443 to masquerade as benign outbound HTTPS traffic, challenging basic port-based firewall heuristics.

3. Execute a stealthy DNS hijacking maneuver. The compromised camera will seamlessly shift its domain name resolution requests to a malicious C2 domain (e.g., `c2.malicious-exfil.com`) while strictly maintaining its baseline 60-second polling frequency. This demonstrates an advanced evasion tactic designed to bypass purely volumetric anomaly detection by preserving statistical temporal normality.

4.4.2 Expected Behaviour and Theoretical Baselines

To ensure statistical significance and provide a robust dataset for comparative feature extraction, Scenario 3 has similarly been executed over a 1200-second (20-minute) capture period.

Unlike the highly visible, noisy volumetric attacks observed in Scenario 2, this scenario expects to demonstrate stealthy, post-exploitation behavior. The anticipated forensic deviations from the control baseline are:

- **Volumetric Camouflage (Flow Density Deception):** Unlike the massive flow spikes caused by the prior network scan and brute-force attacks, Scenario 3 is engineered to evade basic volumetric anomaly detection. Because the attacker only sends a single trigger payload at $t = 300$, and the compromised camera simply redirects its existing bandwidth rather than creating new noise, the aggregate network flow density (flows per minute) is expected to falsely appear as a stable, healthy baseline. The main difference will be that the flow will be directed to a different port and domain.
- **Protocol Masquerading (Port Inversion):** Following the payload trigger, the camera will terminate its authorized local video stream (UDP port 50005) and initiate a new data exfiltration stream. To bypass egress firewall filtering, this massive volumetric stream will target port 443. This will severely corrupt the network's protocol-to-port expectations, logging an anomalous, high-bandwidth UDP stream operating over a port traditionally reserved for TCP-based TLS traffic.
- **Evasive Application-Layer Manipulation (DNS Hijacking):** The compromised camera will maintain its mathematically rigid 60 seconds Inter-Arrival Time (IAT) for DNS queries, further masking the attack from temporal flow analysis. However, it is expected to resolve a total different domain name, the C2 one in this case.
- **External Adjacency Matrix Violation (MUD Corruption):** The Topology Interaction Heatmap will log a cessation of the camera's heavy video traffic to the Gateway, instantly replaced by a massive new density cluster directed toward the unauthorized rogue cloud IP (`203.0.113.5`). This provides absolute visual proof of a critical Manufacturer Usage Description (MUD) profile breach.

4.4.3 Evidence Integrity and Feature Extraction

The same steps of 4.2.3 have been followed to guarantee the integrity and validity of the extracted features.

MUD Profile Validation

The first step is to compare again the expected baseline behaviour to the observed empirical results with the objectivo to detect attacker activity.

Traffic Classification Profile	Expected	Empirical	Status / Delta
Authorized IoT Services			
NTP Synchronization (UDP/123)	6	6	✓ Baseline Match
MQTT Keep-Alives (TCP/8883)	121	120	✓ Baseline Match
MQTT Telemetry Payloads (TCP/8883)	5	5	✓ Baseline Match
Video Feed Stream (to Gateway/50005)	1	1	✓ Terminated Early
TLS Cloud Telemetry (TCP/443)	5	5	✓ Baseline Match
Adversarial & Post-Exploitation Activity			
DNS Queries (UDP/53)	42	58	× Hijacked C2 Polling (+15)
Attacker Trigger Payload (TCP/443)	0	1	× Compromise Event
UDP Data Exfiltration (to Rogue/443)	0	1	× Egress MUD Violation
Zeek QUIC Protocol Anomalies	0	1	× Masquerading Detected
Environmental Noise / Infrastructure			
Background ICMP & Packet Filter Notes	0	2	∼ Minor Hypervisor Noise
Total Connection Flows	**180**	**199**	**Stealth Profile (+10.5%)**

Table 6: Scenario 3: Comparative Traffic Breakdown (Stealth Exfiltration & C2 Routing)

As expected, attacker activity has been registered, though characterized by extreme stealth. In contrast to the 3,459% spike observed in Scenario 2, the data exfiltration attack generated only a 10.5% volumetric increase (199 total connections). Because the attacker utilized a single TCP trigger payload and maintained the camera's baseline DNS polling frequency, the aggregate network density failed to trigger standard volumetric thresholds.

This minimal structural deviation proves that while legitimate device operations continue, sophisticated post-exploitation tactics can seamlessly blend into the authorized traffic baseline. This provides possible justification for application aware security engines: without analyzing the deep packet metadata (such as port 443 protocol masquerading and DNS domain anomalies), the exfiltration stream can remain functionally invisible to traditional network monitors.

Camera Behaviour: Exfiltration & Port Masquerading

The observed transmission behavior of the Smart Camera (Figure **??**) highlights the critical danger of post-exploitation data exfiltration. At a macro volumetric level, the camera's total outbound throughput does not change, avoiding the asymmetric throughput anomalies that trigger basic firewalls.

Instead, the anomaly is entirely topological. The graph clearly highlights the operational shift at $t = 300$ (05:00). Following the attacker's trigger, the camera completely ceases its authorized UDP transmission to the local Gateway on port 50005. Immediately, an equivalent volume of high-density UDP traffic is spun up and redirected to the external rogue cloud infrastructure. By pushing this exfiltration stream through port 443, the attacker successfully camouflages the hijacked video feed, violating the device's zero-trust MUD profile without alerting pure bandwidth-threshold monitors.

DNS Resolution and C2 Polling

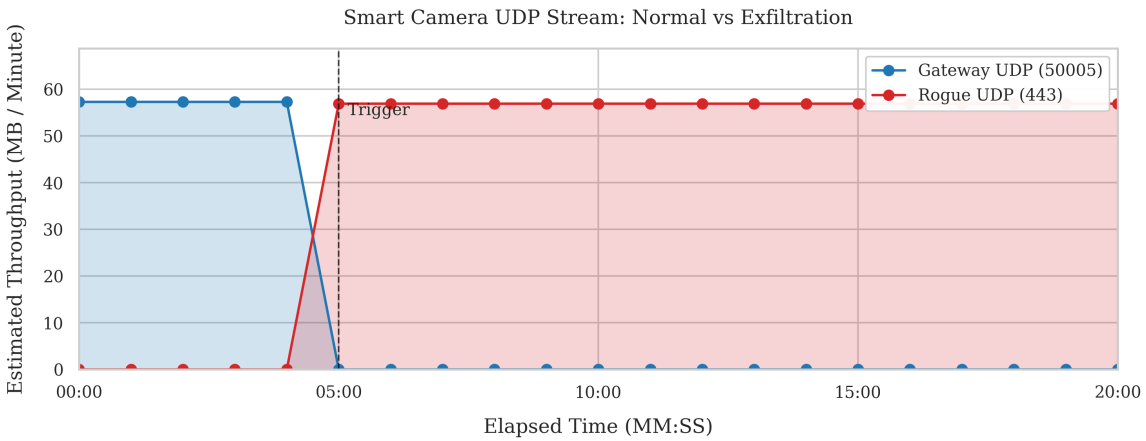

Figure 20: Scenario 3: Throughput change of destination without bandwidth alteration.

In addition to the port masquerading, the attack successfully altered the device's domain name resolution behavior. The empirical data recorded a total of 57 DNS queries, exceeding the zero-trust baseline expectation by 15 requests.

This numerical deviation maps precisely to the attack timeline. Following the compromise at the 5-minute mark, the camera began resolving an unauthorized Command and Control (C2) domain alongside its standard operations. By executing these malicious queries at a steady 60-second interval over the remaining 15 minutes of the capture, the attacker generated exactly 15 anomalous DNS requests. This can highlight a possible vulnerability since without application-layer visibility into the specific queried domains, the device's lateral connection to the adversarial infrastructure remains entirely hidden.

Thermostat Behaviour

The Smart Thermostat sees no changes in behaviour since it is not the target of the attack.

Other Metrics

Figure 21 visualizes the global TCP connection health during the Scenario 3 exfiltration attack. The most critical finding in this data is the total absence of structural anomalies.

Compared to the Scenario 2 brute-force attack flooded the network with over 6,000 `REJ` (Rejected) connections, the data here shows exactly 131 TCP flows, all of which successfully terminated with a normal `SF` state. Because the attacker utilized a single, targeted TCP payload to trigger the UDP exfiltration stream, no unauthorized ports were probed, and the IoT edge devices issued zero TCP Reset (RST) packets. By keeping the network's handshake equilibrium identical to the authorized Scenario 1 baseline, the adversary successfully bypassed structural anomaly detection, proving that severe MUD violations can occur within a mathematically healthy TCP footprint.

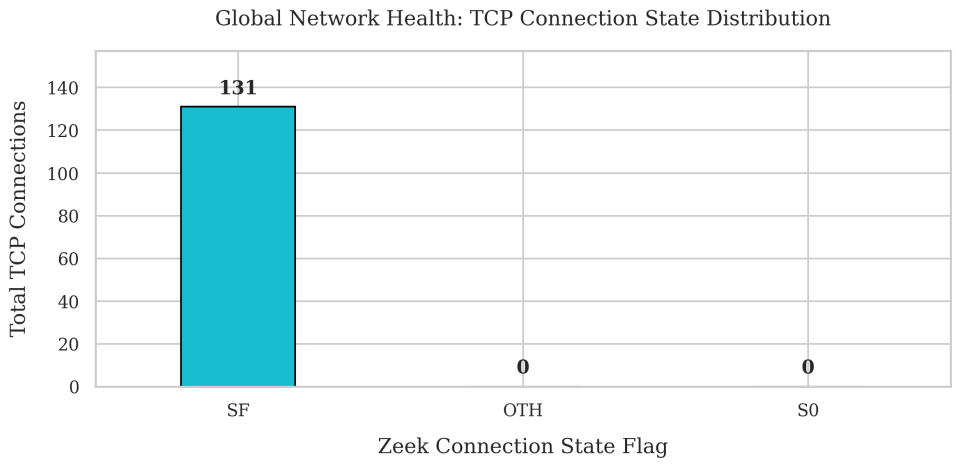

Figure 21: Scenario 3: TCP Connection State Distribution.

Finally, the matrix perfectly captures the authorized baseline behavior, with the Thermostat communicating exclusively with the Gateway (134 connections) and the Camera maintaining its primary authorized link (61 connections). However, the zero-trust architecture is broken by two highly specific, unauthorized flows. First, the matrix logs exactly one connection from the Attacker (`10.0.0.3`) to the Camera, corresponding to the initial trigger payload. Immediately resulting from this compromise, the matrix records one new connection from the Camera to the external Rogue Cloud (`203.0.113.5`). This single external flow definitively proves the MUD violation, capturing the precise path of the hijacked data exfiltration stream.

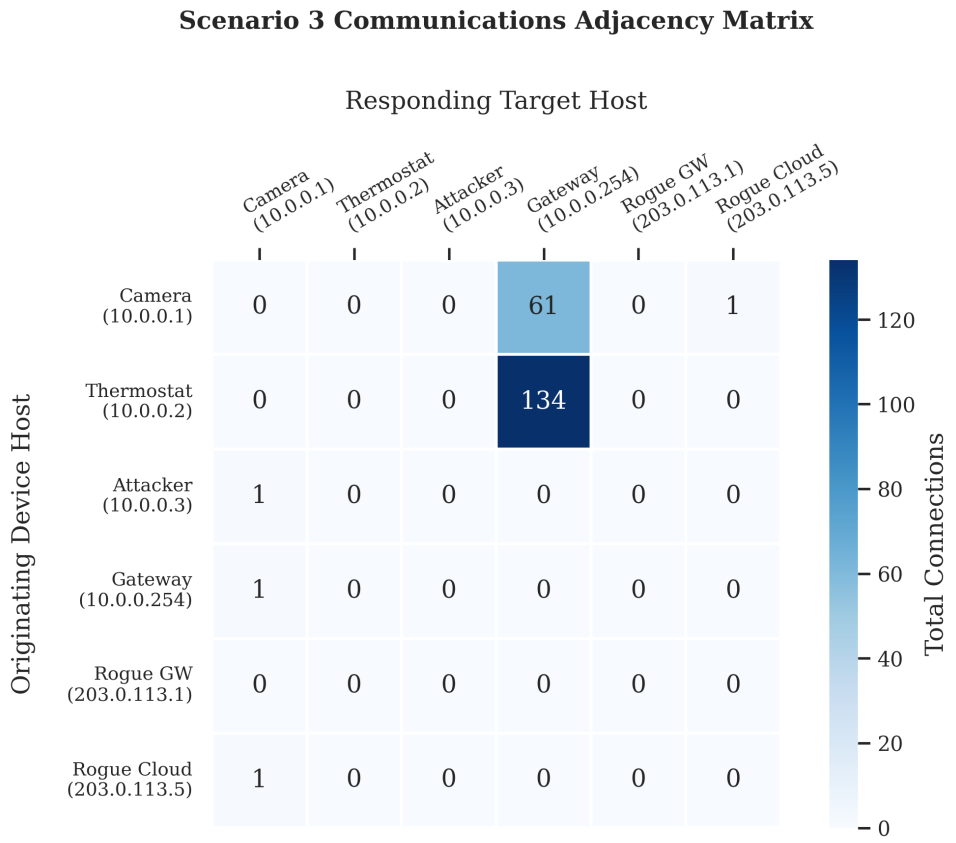

Figure 22: Scenario 3: Empirical interaction heatmap detailing authorized internal network flows.

4.5 Scenario 4: Botnet Infection

4.5.1 Introduction and Objectives

Scenario 4 represents the climax of the adversarial progression, shifting the threat model from stealthy post-exploitation (Scenario 3) to active weaponization and botnet conscription. In this phase, the compromised Smart Thermostat is fully hijacked by the internal attacker node (`10.0.0.3`) and repurposed as an offensive launchpad. To accurately model a distributed malware ecosystem such it could be Mirai, two new external entities are introduced: a malicious Command and Control (C2) infrastructure (`203.0.113.10`) and an external DDoS victim (`203.0.113.99`). The primary objective is to measure and document the catastrophic structural and volumetric deviations caused by lateral propagation and outbound denial-of-service traffic against the Scenario 1 control baseline. The scenario goals in detail are the following:

1. Execute the initial infection and C2 check-in sequence. At a precise temporal marker ($t = 300$ seconds), the internal attacker delivers a targeted payload to the Smart Thermostat. This triggers an immediate DNS resolution of the malicious botnet domain and establishes a persistent, unauthorized TCP control channel (e.g., port 6667) to the external C2 server, violating outbound routing constraints.
2. Demonstrate lateral propagation and internal zero-trust boundary violations. Between $t = 400$ and $t = 600$ seconds, the compromised thermostat will attempt to infect adjacent devices by aggressively scanning the local `10.0.0.0/24` subnet (targeting common IoT ports such as 23 and 80). Forensically, this will capture a massive structural inversion in the TCP state distribution (`REJ` and `S0` flags) originating directly from a trusted edge device rather than the initial attacker node.
3. Launch a volumetric Distributed Denial of Service (DDoS) assault. At $t = 600$ seconds, triggered by a command from the C2 server, the thermostat will initiate a massive, high-bandwidth UDP flood targeting the external victim IP. This goal documents the total saturation of the network's outbound throughput and provides a definitive visual signature of a device completely abandoning its authorized Manufacturer Usage Description (MUD) profile to participate in a global botnet.

4.5.2 Expected Behaviour and Theoretical Baselines

To ensure statistical validity and provide a robust empirical dataset for anomaly detection models, Scenario 4 has similarly been executed over a 1200-second (20-minute) capture period.

Unlike the stealthy exfiltration tactics observed in Scenario 3, this scenario models the aggressive, highly visible progression of a botnet infection. The anticipated forensic deviations from the control baseline are distinctly multi-phased:

- **Egress MUD Violation (C2 Check-in):** The initial compromise at $t = 300$ will immediately fracture the Smart Thermostat's operational baseline. Unlike its authorized, low-frequency MQTT traffic, the device is expected to initiate anomalous outbound DNS queries to resolve the malicious C2 domain, followed by the establishment of a persistent, unauthorized TCP connection (port 6667) to the external botnet infrastructure.
- **Internal Adjacency Violation (Lateral Propagation):** Between $t = 400$ and $t = 600$, the zero-trust isolation boundaries will be catastrophically broken. Forensically, this will manifest on the Topology Interaction Heatmap as an explosive burst of unauthorized lateral traffic originating from the Thermostat and targeting adjacent local IPs. Because this scan targets closed ports, it is expected to generate a massive surge in `REJ` and `S0` TCP connection states, mirroring the structural damage observed in Scenario 2, but uniquely originating from an internal edge asset.

- **Catastrophic Throughput Inversion (The DDoS Assault):** At the $t = 600$ mark, the environment's volumetric baseline will be entirely shattered. The Thermostat—a device whose theoretical baseline dictates payloads no larger than 27 bytes—will initiate a massive 50 Mbps UDP flood directed at the external victim IP. This will create an undeniable, exponential vertical spike in both aggregate network throughput and flow density, effectively weaponizing the device and confirming its conscription into the botnet.

4.5.3 Evidence Integrity and Feature Extraction

The same steps of 4.2.3 have been followed to guarantee the integrity and validity of the extracted features.

MUD Profile Validation

The first step is to compare again the expected baseline behaviour to the observed empirical results with the objectivo to detect attacker activity.

Traffic Classification Profile	Expected	Empirical	Status / Delta
Authorized IoT Services			
Authorized DNS Queries (UDP/53)	31	31	✓ Baseline Match
NTP Synchronization (UDP/123)	5	5	✓ Baseline Match
MQTT Keep-Alives (TCP/8883)	47	47	✓ Baseline Match
MQTT Telemetry Payloads (TCP/8883)	2	2	✓ Baseline Match
Camera Video Feed (UDP/50005)	1	1	✓ Baseline Match
Camera TLS Telemetry (TCP/443)	5	5	✓ Baseline Match
Adversarial & Botnet Activity			
Malicious C2 DNS Lookups (`c2.botnet.com`)	0	3	× Botnet Domain Query
C2 Connections (TCP/6667)	0	18	× Persistent IRC Control Pipe
Lateral Scan Flows (`Thermostat` → Local)	0	128	× Subnet Scan (128 `REJ`)
Volumetric UDP Flood (`Thermostat` → Victim)	0	1	× Exfiltration / DDoS Assault
Environmental Noise / Infrastructure			
Background ICMP Noise (`Gateway` → `Camera`)	0	1	∼ Network Overhead
Total Connection Flows	**91**	**210**	**Structural Disruption (+130.8%)**

Table 7: Scenario 4: Comparative Traffic Breakdown (Thermostat Conscription and 7.47 Mbps Volumetric DDoS Campaign)

Table 7 presents the empirical traffic profile for Scenario 4, tracking the operational transition of the Smart Thermostat from a low-power endpoint to an offensive botnet node. The data illustrates a severe multi-phase compromise that leaves structural indicators across multiple layers of the network stack. While authorized baseline services—such as the 47 MQTT keep-alives and 5 camera TLS flows—remained perfectly aligned with expected control values, adversarial deviations completely altered the environment's global topology.

The structural metrics isolate three clear malicious phases following the initial compromise. First, the application-layer is compromised via 3 unauthorized C2 DNS resolutions targeting `c2.botnet.com`, immediately followed by 18 persistent TCP check-ins on port 6667, confirming active Command and Control conscription. Second, the breach of lateral zero-trust boundaries is mathematically verified

by 128 local scan flows originating from the Thermostat. Because adjacent hosts rejected these unauthorized authentication attempts, 100% of these connection attempts terminated with an explicit `REJ` state flag.

The most important aspect is the presence of a Volumetric UDP flood, which indicates the presence of a possible data exfiltration or DDoS assault as a botnet.

Camera Behaviour
The observed activity from the Smart Camera matches the MUD, since the attack does not target its bandwidth or other messages.

Thermostat Behaviour
The temporal stability of the environment during the pre-infection phase ($t = 0$ to $t = 300$ seconds) is verified by the Smart Thermostat MQTT Keep-Alive Inter-Arrival Time (IAT) distribution, illustrated in Figure 23. During this initial observation window, the device exhibits a highly deterministic, mathematically rigid operational profile. The empirical data demonstrates an uncorrupted telemetry sequence where every single MQTT keep-alive pulse aligns perfectly with time of exactly 10.00 seconds, as expected. In addition, the telemtry size of packets has been seen unaltered during the whole sequence since the attacker does not target this behaviour of the thermostat.

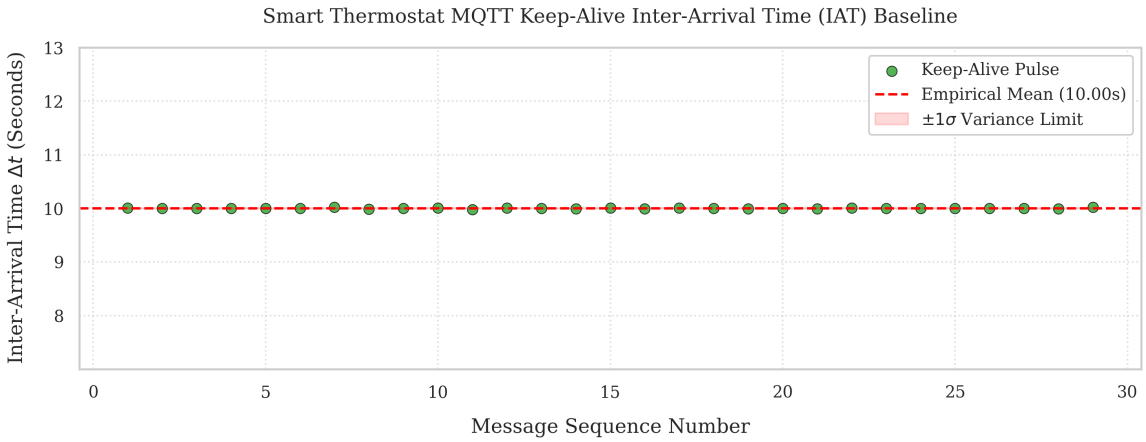

Figure 23: Scenario 4 Normal Phase: IAT distribution during the first 300s of experiment.

However, there are some indicators that show evidence of the existence of a possible botnet infection by the thermostat. First of all, the device performs 3 DNS requests to a domain `c2.botnet.com` which is not in its MUD. This domain is from the C2, which is used to coordinate the infection and attack. Then, immediately following these unauthorized domain resolutions, the thermostat initiates a series of outbound TCP connections on destination port 6667. In network forensics, this specific port is highly significant as it is the default standard for Internet Relay Chat (IRC) communications. This protocol is used by modern IoT modern families due to its highly efficient asynchronous command-broadcastign architecture. These 18 check-ins represent active beaconing intervals where the compromised smart thermostat polls the remote IRC infrastructure to receive operational instructions from the botmaster, establishing absolute empirical proof of an ongoing Command and Control (C2) session that completely violates the device's operational baseline.

Following the successful establishment of the Command and Control channel, the compromised Smart Thermostat transitions from a passive beacon to an active insider threat. This behavioral shift is exemplified in the Lateral Scan Report, which documents a total of 128 anomalous internal flows originating from the node (`10.0.0.2`) and propagating across the local subnet.

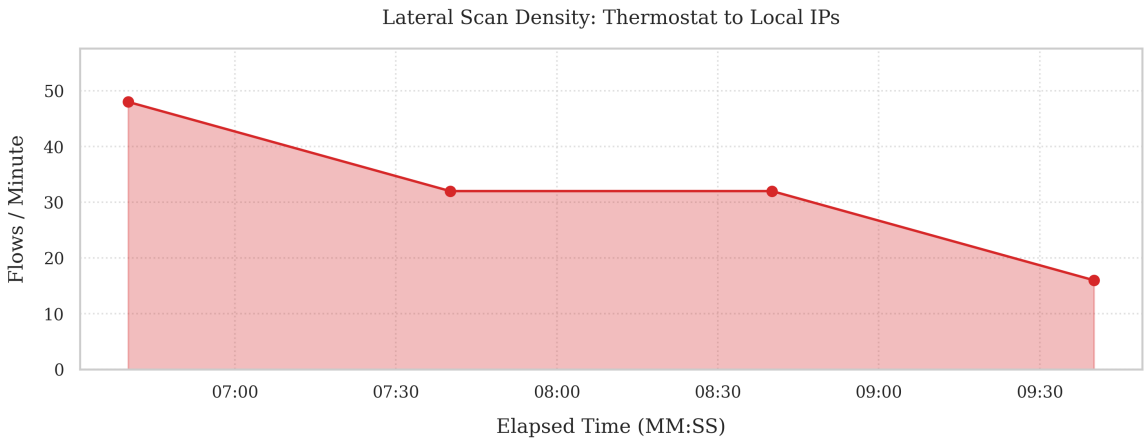

Figure 24: Scenario 4 Internal Propagation Phase: Temporal flow density of the 128 unauthorized lateral scanning attempts originating from the infected Smart Thermostat.

An extended examination of the transport-layer state distribution reveals that 100% of these 128 malicious flows terminated in a `REJ` (Rejected) status, with zero instances of completed handshakes (`SF`) or unanswered stealth attempts (`S0`). This absolute uniformity carries two critical forensic implications.

This fact confirms that the botnet malware was executing an aggressive, automated horizontal scan to locate adjacent network endpoints. This phase creates a massive behavioural anomaly, as a consumer IoT thermostat possesses no legitimate operational reason to broadcast hundreds of connection requests to neighbouring local hosts.

The culmination of the Scenario 4 botnet infection is the execution of a volumetric Distributed Denial of Service (DDoS) assault, directed at the external target infrastructure (`203.0.113.99`). Figure 25 visually maps the temporal throughput of the compromised Smart Thermostat, explicitly demarcating the transition from localized reconnaissance to active weaponization.

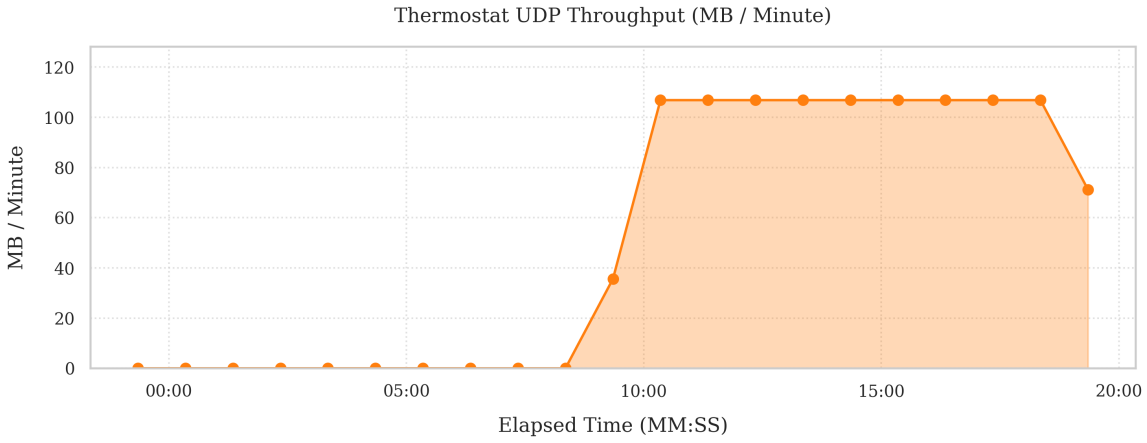

Figure 25: Scenario 4 Volumetric Assault Phase: Sustained UDP flood throughput originating from the compromised thermostat.

During the first half of the capture window, the UDP throughput remains flat at zero, corroborating the established baseline where the device communicates exclusively via TCP-based MQTT keep-alives, following the established MUD behaviour. However, following the C2 check-in and internal lateral scanning phases, the botnet master issues the attack directive. At approximately the 09:30 mark, the graph illustrates a violent architectural deviation, the thermostat begins generating spoofed UDP

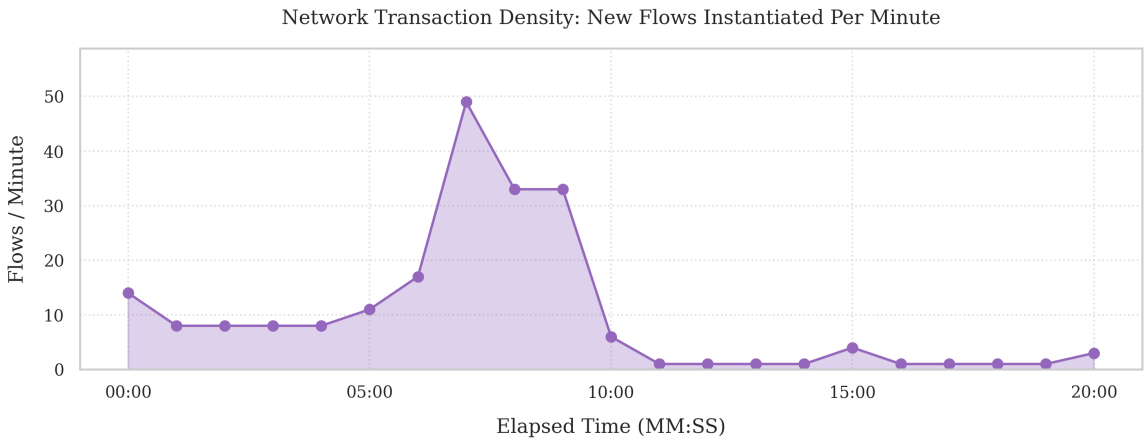

Figure 26: Scenario 4: Network flow density, where a peak can be observed during the network scan.

datagrams. The attack rapidly reaches a sustained throughput, pushing over 100 Megabytes per minute.

Other Metrics

The TCP connection state distribution also provides clear structural proof of the attack's impact on the network. As seen in Scenario 1, in a stable baseline, TCP flows primarily resolve with an SF (Normal) state flag, indicating successful connections and clean teardowns. While authorized IoT traffic continued normally during the attack (registering 39 `SF` flows), it was completely eclipsed by a surge in anomalous activity.

Specifically, Zeek recorded 128 flows terminating in a `REJ` (Rejected) state. This spike in rejected handshakes is the direct footprint of the attacker (`10.0.0.3`) scanning the network. As the scanner flooded the IoT edge devices with `SYN` probes targeting closed ports, the devices continuously issued RST packets to block the unauthorized access.

Consequently, this sudden inversion of the `SF`-to-`REJ` ratio serves as an ideal, automated tripwire for detecting an active internal breach.

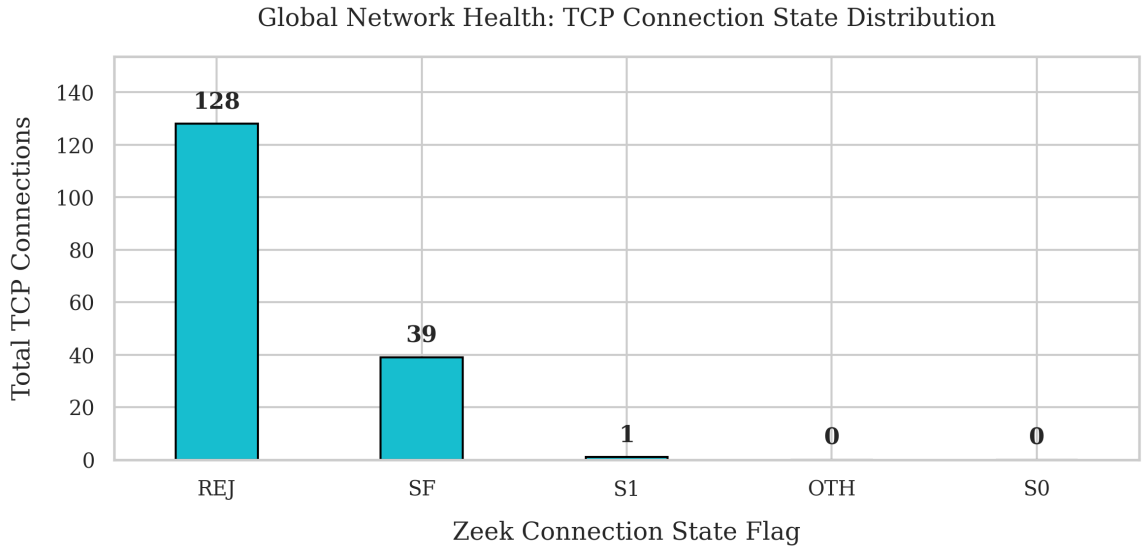

Figure 27: Scenario 4: TCP Connection State Distribution.

Finally, the interaction heatmap is totally altered respect the normality. In this scenario, the baseline has been broken not by external probes, but by the compromised Smart Thermostat (`10.0.0.2`), which now acts as the active origin for the anomalous lateral traffic. The matrix clearly visualizes this internal propagation phase: the infected thermostat launches a horizontal sweep, distributing exactly 64 unauthorized connection attempts to the Smart Camera (`10.0.0.1`) and 64 attempts to the local Attacker node (`10.0.0.3`), corroborating the 128 rejected flows previously analyzed. This internal aggression occurs alongside expected baseline routing, such as the Camera's 30 authorized connections to the Gateway, perfectly illustrating how the compromised IoT endpoint camouflages its lateral botnet scanning amidst standard operational noise.

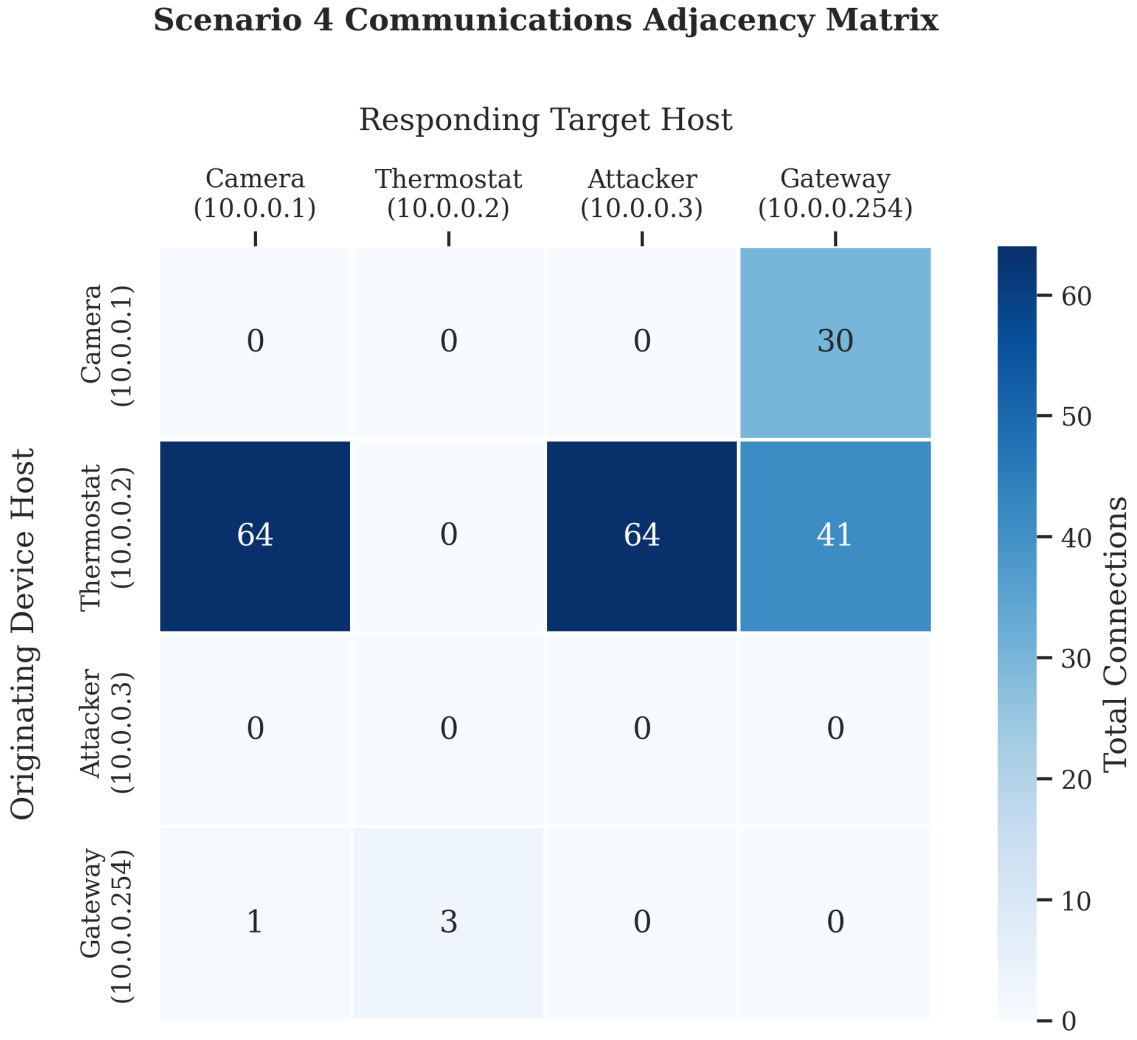

Figure 28: Scenario 4: Empirical interaction heatmap detailing authorized internal network flows.

4.6 Cross-Scenario Comparative Analysis

This section synthesizes the empirical data collected across the four experimental phases. By evaluating the smart home network's operational baseline against a set of different attacks scenarios, we can contrast the structural and behavioural footprint of each attack vector. The comparative analysis highlights a critical reality in IoT network forensics. **The severity of a network compromise does not strictly correlate with flow volume**. Adversaries adapt their techniques and procedures, ranging from massive volumetric spikes to highly evasive port masquerading, based on their specific operational objectives.

4.6.1 The Macro-Statistics and Detection Difficulty Matrix

To establish a view of the threat landscape, the primary network metrics for all four scenarios have been aggregated. Table 8 illustrates how standard flow analysis succeeds against noisy attacks but fails against advanced stealth tactics.

Scenario	Threat Profile	Total Flows	Volumetric Δ	Primary Forensic Indicator
Scenario 1	Operational Baseline	181	0.0%	100% adherence to MUD profiles.
Scenario 2	Recon & Brute-Force	5,987	+3,329%	Massive global flow density surge.
Scenario 3	Stealth Exfiltration	199	+10.5%	DNS anomalies and Port 443 Masquerading.
Scenario 4	Botnet Infection	210	+130.8%	Internal lateral scanning and outbound UDP flood.

Table 8: Macro-Statistics and Detection Difficulty Matrix across all experimental scenarios.

4.6.2 Volumetric Visibility vs. Architectural Stealth

The most striking contrast in the experimental data is the volumetric disparity between the different attack methodologies.

Both Scenario 2 and Scenario 4 performed an automated subnet scan to map the environment. Because network scanners are noisy by flooding targets with thousands of simultaneous `SYN` probes, this attack is highly visible and easily caught by rudimentary threshold-based intrusion detection systems.

Conversely, Scenario 3 demonstrates the danger of sophisticated post-exploitation tactics designed to bypass volumetric heuristics. By relying on a single targeted payload to trigger the malware, the attacker generated a negligible 10.5% increase in total flows (199 connections). Rather than creating new network noise, the compromised Smart Camera simply redirected its existing high-bandwidth UDP video stream to an unauthorized external IP. Because the aggregate network throughput remained seemingly stable, this scenario proves that deep packet metadata inspection—specifically looking for port masquerading and hijacking DNS polling frequencies—is strictly required to detect advanced exfiltration.

4.6.3 The TCP Connection State Tripwire

The distribution of TCP connection states proved to be one of the most reliable automated tripwires for detecting lateral movement within the zero-trust architecture.

In the secure Scenario 1 baseline, the environment was heavily dominated by the `SF` (Normal) state flag, indicating successful connections and clean teardowns. However, when an adversary attempts to breach internal boundaries, this ratio violently inverts:

- **External Lateral Scanning (Scenario 2):** As the attacker node flooded the IoT edge devices with unauthorized probes, the devices actively defended themselves by issuing TCP Reset (RST) packets, resulting in an explosive 6,041 flows terminating in a `REJ` (Rejected) state.
- **Internal Insider Threat (Scenario 4):** A similar structural inversion occurred when the Smart Thermostat was conscripted into a botnet. As the hijacked thermostat aggressively scanned the local `10.0.0.0/24` subnet, Zeek recorded 128 malicious flows terminating in a `REJ` status.

Crucially, Scenario 3 entirely bypassed this tripwire. Because the attacker utilized a single payload and did not indiscriminately probe closed ports, the network generated zero `REJ` states, maintaining an illusion of a mathematically healthy TCP footprint with exactly 132 `SF` flows.

4.6.4 The Degradation of Zero-Trust and MUD Profiles

The progression from Scenario 1 to Scenario 4 effectively maps the degradation of the Manufacturer Usage Description (MUD) profiles under increasing adversarial pressure.

The control environment established a strict node-to-node isolation matrix. Scenario 2 violated this via unauthorized probes directed at the devices. Scenario 3 bypassed these rules via egress manipulation, utilizing DNS hijacking and routing stolen video feeds to an unauthorized external rogue cloud. Finally, Scenario 4 represented a total collapse of both ingress and egress trust boundaries, as the Smart Thermostat established an unauthorized IRC-style Command and Control link and weaponized its throughput to participate in a global DDoS campaign.

5 Conclusions

The primary objective of this research projct was to address the existing limitations of traditional digital forensics in IoT environments by developing a robust, passive network-based methodology for behavioral reconstruction. Driven by the volatility of physical device memory, the reliance on proprietary hardware, and the widespread adoption of encryption, this project hypothesized that a device's physical state and malicious compromise could be definitively determined solely from encrypted network metadata.

By systematically evaluating a simulated smart home topology against a baseline of authorized Manufacturer Usage Description (MUD) behaviours and three escalating adversarial scenarios, the research successfully bridged the semantic gap between raw network traffic and physical device actions.

5.1 Fulfillment of the Research Questions

The findings of this project directly answer the core research questions established at the onset of the study:

1. **Can IoT device activity be inferred solely from network traffic?**
   The experimental results definitively prove that device activity can be inferred without payload decryption or physical device access. By establishing a rigid temporal and volumetric baseline (Scenario 1), any deviation of it was successfully isolated and identified using purely contextual flow metadata. Whether if the attack was a high-frequency credential brute force attack (Scenario 2) or the weaponization of a thermostat into a botnet (Scenario 4).

2. **Which network features best identify device behavior?**
   The research identified that the severity and type of an attack dictate the required forensic feature. For noisy, automated attacks (Reconnaissance and DDoS), **global flow density** and **TCP connection state distributions** (specifically `SF` and `REJ` flags) served as immediate, high-fidelity tripwires. Conversely, for advanced persistent threats (Stealth Exfiltration), volumetric metrics failed entirely. Detecting these attacks required application-layer metadata, specifically **DNS query frequency** (identifying C2 domain polling) and **protocol-to-port masquerading** (detecting UDP streams hiding over TCP/443).

3. **How accurately can forensic timelines be reconstructed?**
   The methodology demonstrated millisecond accuracy in chronological timeline reconstruction. By mapping volumetric anomalies (e.g., the sudden 7.47 Mbps UDP flood) and structural anomalies (e.g., the abrupt cessation of authorized port 50005 traffic) back to the theoretical data flow models (NIST SP 800-183), the investigation successfully tracked the exact moment of initial payload execution, lateral propagation, and final data exfiltration.

5.2 Expectations vs. Empirical Reality: The Zero-Trust Paradox

The most critical finding of this study emerged from evaluating the effectiveness of the theoretical zero-trust architecture and NIST MUD profiles against active exploitation.

What was expected: It was expected that implementing strict zero-trust network boundaries and MUD profiles would isolate compromised devices, preventing them from interacting with unauthorized internal nodes or external servers.

What was observed: The architecture succeeded at *ingress* containment but suffered a catastrophic failure at *egress* containment.

During Scenario 2 and Scenario 4, the strict internal access control lists performed perfectly. When the attacker and the compromised device attempted to scan the local subnet, the target devices actively rejected the unauthorized requests, generating thousands of `REJ` states and successfully halting lateral propagation.

However, in Scenario 3 and Scenario 4, the compromised edge devices effortlessly bypassed the network perimeter to communicate with external Command and Control (C2) servers, exfiltrate data, and launch a global DDoS assault. Because edge devices require internet access to function, the perimeter firewall implicitly trusted their outbound connections. This exposes a severe vulnerability in current IoT network design: static MUD profiles are inherently trust-biased toward the internal endpoint.

Furthermore, the data debunked the assumption that all cyberattacks generate massive network noise. While the brute-force attack (Scenario 2) generated a massive 3,329% spike in traffic, the data exfiltration attack (Scenario 3) generated a nearly invisible 10.5% volumetric increase. The attacker successfully hijacked the camera's existing bandwidth, proving that modern IoT malware is designed to camouflage itself within the authorized volumetric baseline.

5.3 Contributions to the Field

This project contributes a validated, end-to-end framework for IoT network forensics. By moving away from Deep Packet Inspection (DPI) and relying on Zeek-extracted metadata, the methodology remains resilient against modern cryptographic standards like TLS 1.3 and DTLS. Additionally, this research provides a fully documented, open-source dataset of labeled PCAP files and Zeek logs representing both benign IoT MUD compliance and multi-stage botnet kill chains, which can be utilized by the academic community to train automated Intrusion Detection Systems (IDS).

5.4 Future Work

While the simulated Mininet environment provided a mathematically pristine testing ground, future research must validate these findings against physical hardware. Transitioning this methodology from virtualized emulators to commercial devices will introduce real-world proprietary background noise, allowing for the refinement of the feature extraction pipeline.

References

[1] Yasar Kinza Gillis, Alexander S. What is the Internet of Things (IoT)? TechTarget IoT Agenda, 2025. Accessed: April 12, 2026.

[2] WinSystems. Cloud, Fog, and Edge Computing: What's the Difference?, 2020. Accessed: April 12, 2026.

[3] Mahmoud A. M. Albreem, Abdul Manan Sheikh, Mohammed J. K. Bashir, and Ayman A. El-Saleh. Towards green Internet of Things (IoT) for a sustainable future in Gulf Cooperation Council countries: current practices, challenges and future prospective. *Wireless Networks*, 28:1–25, 2022.

[4] Manubes. What is MQTT? A Complete Guide, 2023. Accessed: April 22, 2026.

[5] Michael Fagan, Katerina Megas, Karen Scarfone, and Matthew Smith. NISTIR 8259a: IoT Device Cybersecurity Capability Core Baseline. Nist interagency or internal report, National Institute of Standards and Technology (NIST), May 2020.

[6] University of New South Wales. IoT Sentinel and MUD Profiles Repository, 2024. Accessed: May 6, 2026.

[7] Mohd Javaid, Abid Haleem, Ravi Pratap Singh, and Rajiv Suman. A review on Internet of Things (Iot), Machine Learning (ML), and Deep Learning (DL) in Smart Agriculture. *Sensors*, 22(3):995, 2022.

[8] OWASP Foundation. OWASP Internet of Things Project: Iot top 10, 2018. Accessed: April 12, 2026.

[9] Karen Kent, Suzanne Chevalier, Tim Grance, and Hung Dang. Guide to Integrating Forensic Techniques into Incident Response. Technical Report NIST SP 800-86, National Institute of Standards and Technology (NIST), Gaithersburg, MD, 2006.

[10] Mansour Alenezi, Yunus Al-Nidawi, and Abdullah Al-Zahrani. Internet of Things (Iot) security: A comprehensive review of forensic analysis, challenges, and future directions. *Computer Science Review*, 55:100839, 2025.

[11] Áine MacDermott, Thar Baker, Paul Buck, Farkhund Iqbal, and Qi Shi. The internet of things: Challenges and considerations for cybercrime investigations and digital forensics. *International Journal of Digital Crime and Forensics*, 12:1–13, 01 2020.

[12] Becker B. O'Sullivan T. Scanlon M. Lillis, D. Current challenges and future research areas for digital forensic investigation. *The 11th ADFSL Conference on Digital Forensics, Security and Law (CDFSL 2016)*, 04 2016.

[13] International Telecommunication Union. Recommendation ITU-T Y.2060: Overview of the Internet of Things. Recommendation, ITU-T, 2012. Global Information Infrastructure, Internet Protocol Aspects and Next-Generation Networks.

[14] Flavio Bonomi, Rodolfo Milito, Jiang Zhu, and Sateesh Addepalli. Fog Computing and its Role in the Internet of Things. In *Proceedings of the First Edition of the MCC Workshop on Mobile Cloud Computing*, MCC '12, pages 13–16, New York, NY, USA, 2012. Association for Computing Machinery (ACM).

[15] Henk Birkholz, Thomas Fossati, Soumya Panani, and Greg Desruisseaux. RFC 9431: Asset Identification for the Internet of Things (IoT). RFC 9431, Internet Engineering Task Force (IETF), July 2023.

[16] Norulzahrah Hassan, Norziana Mohd Aman, Norziana Bakar, and Shahrulniza Ibrahim. A Forensic Analysis on the Availability of MQTT Network Traffic Artifacts. *Journal of Physics: Conference Series*, 1860(1):012002, 2021.

[17] Zach Shelby, Klaus Hartke, and Carsten Bormann. RFC 7252: The Constrained Application Protocol (CoAP). RFC 7252, Internet Engineering Task Force (IETF), June 2014.

[18] Markus Miettinen, Samuel Marchal, Ibbad Hafeez, N. Asokan, Ahmad-Reza Sadeghi, and Sasu Tarkoma. IoT Sentinel: Automated Device Identification and Damage Mitigation in Smart Homes. In *Proceedings of the 37th IEEE International Conference on Distributed Computing Systems (ICDCS)*, pages 116–127, 2017.

[19] Arunan Sivanathan, Hassan Habibi Gharakheili, Franco Loi, Adam Radford, Chirath Wijenayake, Arun Vishwanath, and Vijay Sivaraman. Classifying IoT Devices in Smart Environments Using Network Traffic Characteristics. *IEEE Transactions on Mobile Computing*, 18(8):1746–1759, 2019.

[20] Michael Fagan, Katerina Megas, Karen Scarfone, and Matthew Smith. NISTIR 8259: Foundational Cybersecurity Activities for IoT Device Manufacturers. Nist interagency or internal report, National Institute of Standards and Technology (NIST), May 2020.

[21] William Fisher et al. NIST SP 1800-15: Securing Small-Business Information Systems: Appropriate Tier 1 Resilience. Nist special publication, National Institute of Standards and Technology (NIST), 2019.

[22] Jeffrey Voas. NIST SP 800-183: Networks of 'Things'. Nist special publication, National Institute of Standards and Technology (NIST), July 2016.

[23] Shams Zawoad and Ragib Hasan. Faiot: Towards building a forensics aware eco system for the internet of things. In *2015 IEEE International Conference on Services Computing*, pages 279–284, 2015.

[24] Victor R. Kebande and Indrakshi Ray. A Generic Digital Forensic Investigation Framework for Internet of Things (IoT). In *2016 IEEE 4th International Conference on Future Internet of Things and Cloud (FiCloud)*, pages 356–362. IEEE, 2016.

[25] Bob Lantz, Brandon Heller, and Nick McKeown. A network in a laptop: rapid prototyping for software-defined networks. In *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks*, pages 1–6, 2010.

[26] Wireshark Foundation. Wireshark: The world's most popular network protocol analyzer. `https://www.wireshark.org/`, 2024. Version 4.2.x.

[27] The Tcpdump Group. tcpdump and libpcap. `https://www.tcpdump.org/`, 2024.

[28] Vern Paxson. Bro: a system for detecting network intruders in real-time. *Computer networks*, 31(23-24):2435–2463, 1999.

[29] Wes McKinney. Data structures for statistical computing in python. In Stéfan van der Walt and Jarrod Millman, editors, *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010.